

"Σειρά Αναλυτικών Ασκήσεων" ΝΝ-Ε

Προσωπικά Στοιχεία

Όνοματεπώνυμο: Πέππος Μιχαήλ-Αθανάσιος

Α.Μ.: 03121026

Εξάμηνο: 9^ο $A \leftarrow \sum x_i, x_i^2$

Email: peppas.michael@gmail.com / el21026@mail.ntua.gr

1. Ταξινόμηση στη μία διάσταση

1. (α) Έχουμε: Κλίση $A: x = \{1, 3, 5\} = \{x_1, x_3, x_5\}$
 Κλίση $B: x = \{2, 4, 6\} = \{x_2, x_4, x_6\}$

Ο ταξινομητής 1-NN θα αποδεχθεί ένα νέο στοιχείο, έστω x' , στην κλίση στην οποία βρίσκεται το κοντινότερο στοιχείο του x' . Η απόσταση (Ευκλείδεια) δίνεται από τον τύπο: $d(x, x') = |x - x'|$, αφού έχουμε 1-d δεδομένα.

Το κοντινότερο στοιχείο προς το x' "αλλάζει", όταν μεταβαίνουμε από το μέσον δύο σημείων εκπαίδευσης προς οποιαδήποτε κατεύθυνση. Παρατηρούμε ότι τα σημεία εκπαίδευσης είναι εναλλάξ και έχουν απόσταση 1. Συνεπώς, τα σημεία απόφασης (μέσα αυτών) είναι τα:

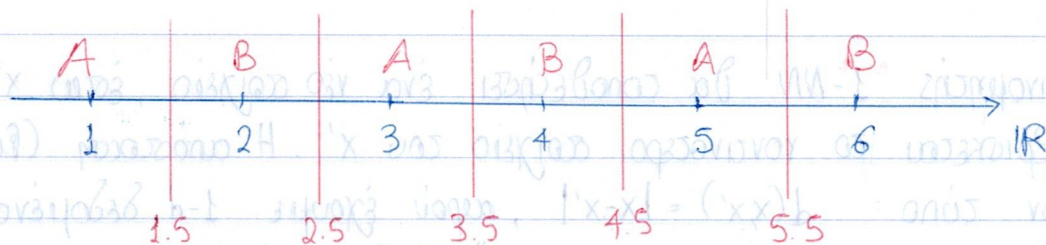
- $(1+2)/2 = 1.5$
 - $(2+3)/2 = 2.5$
 - $(3+4)/2 = 3.5$
 - $(4+5)/2 = 4.5$
 - $(5+6)/2 = 5.5$
- Συνεπώς, έχοντας αυτά τα όρια απόφασης ο 1-NN αναπαράγει για το x' ότι:
- $x' \in A \iff x' \in (-\infty, 1.5) \cup (2.5, 3.5) \cup (4.5, 5.5)$
- $x' \in B \iff x' \in (1.5, 2.5) \cup (3.5, 4.5) \cup (5.5, +\infty)$

Όσον αφορά τον 3-NN, αυτός κάνει "majority voting", δηλαδή αναθέτει το x' στην κλάση της οποίας βρίσκονται 2 εκ των 3 πλησιέστερων σημείων (τουλάχιστον 2). Παράδειγμα, τα παραπάνω σημεία ενδιαφέροντος (μέσα των σημείων εκπαίδευσης) δεν αλλάζουν. Όμως, αλλάζουν οι κλάσεις ταξινόμησης, ως εξής:

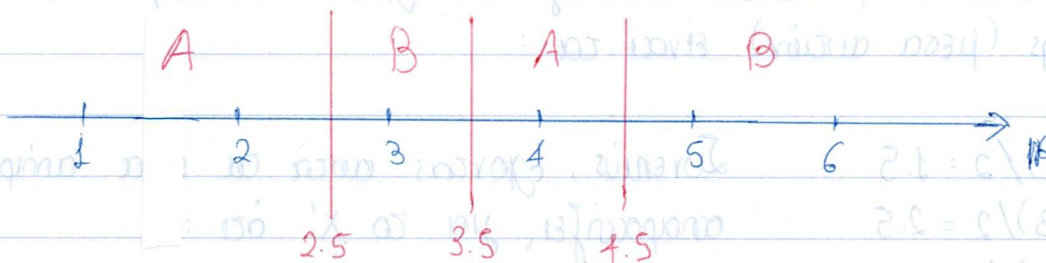
Διάστημα	3-NN	Majority Vote = Classification
$(-\infty, 1.5)$	x_1, x_2, x_3	$\{x_1, x_3\} \Rightarrow A$
$(1.5, 2.5)$	x_1, x_2, x_3	$\{x_1, x_3\} \Rightarrow A$
$(2.5, 3.5)$	x_2, x_3, x_4	$\{x_2, x_4\} \Rightarrow B$
$(3.5, 4.5)$	x_3, x_4, x_5	$\{x_3, x_5\} \Rightarrow A$
$(4.5, 5.5)$	x_4, x_5, x_6	$\{x_4, x_6\} \Rightarrow B$
$(5.5, +\infty)$	x_4, x_5, x_6	$\{x_4, x_6\} \Rightarrow B$

Γραφικά, αυτά φαίνονται ως ακολούθως:

1-NN:



3-NN:



Με μεγάλη, ραβδία, τρέκικη γραμμή είναι τα όρια απόφασης των ταξινομήσεων.

2. (α) Θεωρούμε ότι επεξεργάζομαστε τα δείγματα σε αλυσίδα σειρά, ως εξής:

$$(1, A) \rightarrow (2, B) \rightarrow (3, A) \rightarrow (4, B) \rightarrow (5, A) \rightarrow (6, B)$$

Επίσης, θεωρούμε κωδικοποίηση των κλάσεων-σάχνων ως:

$$\begin{cases} \text{Class A} \Rightarrow t = +1 \\ \text{Class B} \Rightarrow t = -1 \end{cases}$$

Η μορφή του perceptron είναι: $\hat{y} = \text{sign}(wx + b) = \begin{cases} +1, & \text{εάν } wx + b \geq 0 \\ -1, & \text{εάν } wx + b < 0 \end{cases} (*)$

Οι καριές εκπαίδευσης είναι:

$$\begin{cases} w_{\text{new}} = w_{\text{old}} + \eta t x \\ b_{\text{new}} = b_{\text{old}} + \eta t \end{cases}$$

και εφαρμόζονται ΜΟΝΟ εάν $\hat{y} \neq t$,

δηλαδή σε λανθασμένη ταξινόμηση.

Εδώ, το βήμα εκπαίδευσης $\eta = 1$.

Αρχικοποιούμε τις παραμέτρους σε $w^{(0)} = 0$ και $b^{(0)} = 0$.

Συνεπώς, αρχικά $\hat{y} = \text{sign}(w) = +1$, για όλα τα σημεία $\Rightarrow$ (Class A - 1 στο

↳ από σύμβαση (*))

(β) Εκτελούμε τον αλγόριθμο εκπαίδευσης ως :

Σημείο	w_{old}	b_{old}	$\alpha = wx + b$	$\hat{y} = \text{sign}(\alpha)$	t	(Class(t))	w_{new}	b_{new}	Update
(1, A)	0	0	$0 \cdot 1 + 0 = 0$	+1	+1	A	0	0	οχι
(2, B)	0	0	$0 \cdot 1 + 0 = 0$	+1	-1	B	$0 + (-2) = -2$	$0 + (-1) = -1$	ναι
(3, A)*	-2	-1	$-2 \cdot 3 + (-1) = -7$	-1	+1	A	$-2 + 3 = 1$	$-1 + 1 = 0$	ναι
(4, B)*	1	0	$1 \cdot 4 + 0 = 4$	+1	-1	B	$1 + (-4) = -3$	$0 + (-1) = -1$	ναι
(5, A)*	-3	-1	$-3 \cdot 5 + (-1) = -16$	-1	+1	A	$-3 + 5 = 2$	$-1 + 1 = 0$	ναι
* (6, B)	2	0	$2 \cdot 6 + 0 = 12$	+1	-1	B	$2 - 6 = -4$	$0 - 1 = -1$	ναι

Συνεπώς, μετά από μία εποχή: $w^{(1)} = -4$ και $b^{(1)} = -1$.

(c) Όπως αναφέραμε, στο τέλος της 1ης εγχύσης: $w^{(1)} = -4$ και $b^{(1)} = -1$.
 Ο ταξινομητής μας είναι: $\hat{y} = \text{sign}(wx+b) = \text{sign}(-4x-1)$

Είκοθα προκύπτει ότι: $\forall x > 0 \Rightarrow \hat{y} = -1 \Rightarrow \text{class B}$

Συνεπώς, όλα τα σημεία μας ταξινομούνται στην κλάση B:

$\{2, 4, 6\} \Rightarrow B$: σωστό

(* $\{1, 3, 5\} \Rightarrow B$: λάθος!)

Το όριο απόφασης είναι: $wx+b=0 \Leftrightarrow -4x-1=0 \Leftrightarrow x = -1/4$

Άρα: $\begin{cases} x \leq -1/4 \Rightarrow A \\ x > -1/4 \Rightarrow B \end{cases}$

Το παραπάνω πρόβλημα οδηγεί σε λανθασμένη ταξινόμηση, καθώς σε 1-διάσταση το όριο απόφασης είναι απλώς ένα σημείο. Ως εκ τούτου, εάν υπάρχουν σημεία από 2 κλάσεις που "εναλλάσσονται", δηλαδή δεν βρίσκονται όλα δεξιά από ένα σημείο για τη μία κλάση και όλα αριστερά για την άλλη, τότε το πρόβλημα γίνεται μη-γραμμικά διαχωρίσιμο και δεν έχει λύση χωρίς σφάλμα στις ταξινομήσεις. Συνεπώς, το perceptron δεν μπορεί να διαχωρίσει αυτό το πρόβλημα. Μπορεί, μόνο στην ειδική περίπτωση που:

κλάση A = σύνολο σημείων x_A	} $\xrightarrow[\text{διαχωρίσιμο}]{\text{γραμμικά}}$	{ Υπάρχει x^* τ.ώ. :
κλάση B = σύνολο σημείων x_B		
σημείο $x^* \in \mathbb{R}$		$\begin{cases} x' \in x_A \Rightarrow x' > x^* \\ x'' \in x_B \Rightarrow x'' < x^* \end{cases}$

Εδώ, τα σημεία των κλάσεων εναλλάσσονται, άρα το πρόβλημα δεν είναι γραμμικά διαχωρίσιμο.

(d) Όποιους με το ερώτημα (b) έχουμε :

Σημείο	W _{old}	b _{old}	a = wx + b	$\hat{y} = \text{sign}(a)$	t	Class(t)	W _{new}	b _{new}	Update
(1, A)	-4	-1	-5	-1	+1	A	-3	0	ναι
(2, B)	-3	0	-6	-1	-1	B	-3	0	όχι
(3, A)	-3	0	-9	-1	+1	A	0	1	ναι
(4, B)	0	1	1	+1	-1	B	-4	0	ναι
(5, A)	-4	0	-20	-1	+1	A	1	1	ναι
(6, B)	1	1	7	+1	-1	B	-5	0	ναι

$$(+)$$

Τελικώς, μετά τη δεύτερη επόχνη: $W^{(2)} = -5$ και $b^{(2)} = 0$.

Επομένως $\hat{y} = \text{sign}(wx + b) = \text{sign}(-5x + 0)$

Ακόμα ισχύει ότι: $\forall x \geq 0 \Rightarrow \hat{y} = -1 \Rightarrow \text{class B}$

Άρα, το πρόβλημα ακόμα δεν λύνθηκε (τα x_1, x_2, x_5 ταξινομούνται λάθος).

Ο ισχυρισμός στο ερώτημα (c) επιβεβαιώνεται και παραματρικά δοιόν.

$$(2) \left[x \sum_{i=1}^5 \frac{1}{5} = 1 \right] \Rightarrow 0 = (1-x) \sum_{i=1}^5 \frac{1}{5}$$

3. (α) Για κάθε κλάση $K \in \{A, B\}$ υποθέτουμε ότι ο

$$\Rightarrow 0 = (1 - p(x|C)) \sim \mathcal{N}(x | \mu_c, \sigma_c^2) \Rightarrow 0 = \frac{96}{526}$$

$$\text{Για 1-διάσταση: } p(x | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \quad (1)$$

Έστω $\{x_1, x_2, \dots, x_n\}$ i.i.d. . Τότε δέχουμε να μεγιστοποιήσουμε τη συνάρτηση πιθανότητας :

$$L(\mu, \sigma^2) = p(x_1, x_2, \dots, x_n | \mu, \sigma^2) \stackrel{\text{i.i.d.}}{=} \prod_{i=1}^n p(x_i | \mu, \sigma^2) \stackrel{(1)}{=} \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right)$$

Παίρνουμε την log-likelihood και έχουμε:

$$\ell(\mu, \sigma^2) = \ln L(\mu, \sigma^2) = \sum_{i=1}^n \ln(p(x_i; \mu, \sigma^2)) \quad (2)$$

$$\text{Είναι: } \ln p(x_i; \mu, \sigma^2) = -\frac{1}{2} \ln(2\pi\sigma^2) - \frac{(x_i - \mu)^2}{2\sigma^2} \quad (3)$$

$$\text{Άρα: } (2) \stackrel{(3)}{\Rightarrow} \ell(\mu, \sigma^2) = -\frac{n}{2} \ln(2\pi\sigma^2) - \sum_{i=1}^n \frac{(x_i - \mu)^2}{2\sigma^2} \quad \eta$$

$$\ell(\mu, \sigma^2) = -\frac{n}{2} \ln(2\pi) - \frac{n}{2} \ln \sigma^2 - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2 \quad (4)$$

Για να βρούμε την εκτίμηση MLE μιας παραμέτρου, έστω θ , παραγωγίζουμε ως προς θ και θέτουμε την μερική παραγωγή ίση με το 0. Συνεπώς:

$$\begin{aligned} \bullet \text{ Για } \hat{\mu}_{MLE}: \quad \frac{\partial \ell}{\partial \mu} &= 0 \stackrel{(4)}{\Rightarrow} -\frac{1}{2\sigma^2} \sum_{i=1}^n 2(x_i - \mu) \cdot (-1) = 0 \quad (\Rightarrow) \\ \sum_{i=1}^n (x_i - \mu) &= 0 \quad (\Rightarrow) \quad \boxed{\hat{\mu}_{MLE} = \frac{1}{n} \sum_{i=1}^n x_i} \quad (5) \end{aligned}$$

δηλαδή ο μ.ο. των παρατηρήσεών μας

$$\begin{aligned} \bullet \text{ Για } \hat{\sigma}_{MLE}^2: \quad \frac{\partial \ell}{\partial \sigma^2} &= 0 \stackrel{(4)}{\Rightarrow} -\frac{n}{2} \cdot \frac{1}{\sigma^2} \cdot 1 + \frac{1}{2\sigma^4} \sum_{i=1}^n (x_i - \mu)^2 = 0 \quad (\Rightarrow) \\ -\frac{n}{2\sigma^2} + \frac{1}{2\sigma^4} \sum_{i=1}^n (x_i - \mu)^2 &= 0 \quad (\Rightarrow) \quad -n\sigma^2 + \sum_{i=1}^n (x_i - \mu)^2 = 0 \quad (\Rightarrow) \\ \boxed{\hat{\sigma}_{MLE}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2} \quad (6) \end{aligned}$$

Επομένως, για την κλάση A: $\hat{\mu}_A = \frac{1+3+5}{3} = 3$

$$\Rightarrow \sigma_A^2 = \frac{1}{3} \left((1-3)^2 + (3-3)^2 + (5-3)^2 \right) = \frac{8}{3}$$

Άρα: $p(x|A) \sim N(3, 8/3)$.

Ευμενέων, για την κλάση B: $\hat{\mu}_B = \frac{2+4+6}{3} = 4$

$$\Rightarrow \sigma_B^2 = \frac{1}{3} \left((2-4)^2 + (4-4)^2 + (6-4)^2 \right) = \frac{8}{3}$$

Άρα: $p(x|B) \sim N(4, 8/3)$.

4. Ο ταξινομητής Bayes ταξινομεί ένα δείγμα x στην κλάση με τη μεγαλύτερη α posteriori πιθανότητα, δηλαδή:

Bayes $\Rightarrow \begin{cases} \text{Class A, if } P(A|x) > P(B|x) \\ \text{Class B, otherwise} \end{cases} \Leftrightarrow P(x|A)P(A) > P(x|B)P(B)$

Επομένως, το σφάλμα της ταξινόμησης δίνεται από τον τύπο:

$$P(\text{error}) = P(A) \int_{R_B} p(x|A) dx + P(B) \int_{R_A} p(x|B) dx$$

όπου R_A η περιοχή που αναπαριστούμε υπό την κλάση A, και R_B υπό την B.

Αρχικά, βρίσκουμε την επιφάνεια απόφασης (σημείο), η οποία είναι εκεί όπου οι α posteriori πιθανότητες είναι ίσες, δηλαδή:

$$P(A|x) = P(B|x) \Leftrightarrow P(x|A)P(A) = P(x|B)P(B) \Leftrightarrow \ln P(A) - \frac{(x-\mu_A)^2}{2\sigma_A^2} = \ln P(B) - \frac{(x-\mu_B)^2}{2\sigma^2}$$

$$\Leftrightarrow \frac{2\sigma^2 \ln P(A)}{\sigma_A^2} = (x-\mu_A)^2 - (x-\mu_B)^2 = x^2 - 2\mu_A x + \mu_A^2 - x^2 + 2\mu_B x - \mu_B^2 \Leftrightarrow$$

$$2\sigma^2 \ln \frac{P(A)}{P(B)} = \mu_A^2 - \mu_B^2 + 2x(\mu_B - \mu_A) \Leftrightarrow -(\mu_A + \mu_B)(\mu_B - \mu_A) + 2x(\mu_B - \mu_A) \Leftrightarrow$$

$$2\sigma^2 \ln \frac{P(A)}{P(B)} = (\mu_B - \mu_A)(2x - \mu_A - \mu_B) \Leftrightarrow 2x - \mu_A - \mu_B = \frac{2\sigma^2 \ln \frac{P(A)}{P(B)}}{\mu_B - \mu_A} \Leftrightarrow$$

$$\boxed{X^* = \frac{\mu_A + \mu_B}{2} + \frac{\sigma^2 \ln \frac{P(A)}{P(B)}}{\mu_B - \mu_A}} \quad (8)$$

Ενέργεια, η (7) = χαίρεται: (7) $\xRightarrow{(8)}$ $P(\text{error}) = P(\text{error}|A)P(A) + P(\text{error}|B)P(B) \Leftrightarrow$

$$P(\text{error}) = P(X_A > X^*)P(A) + P(X_B < X^*)P(B) \Leftrightarrow$$

$$\boxed{P(\text{error}) = P(A) \left(1 - \Phi\left(\frac{X^* - \mu_A}{\sigma}\right) \right) + P(B) \Phi\left(\frac{X^* - \mu_B}{\sigma}\right)} \quad (9)$$

Λύματα με τα παραπάνω αναντουμε στα ερωτήματα.

(α) Εάν $P(A) = P(B)$: $(8) \Rightarrow X^* = \frac{\mu_A + \mu_B}{2} = \frac{3+4}{2} = 3.5$

$$(9) \Rightarrow P(\text{error}) = \frac{1}{2} \left(1 - \Phi\left(\frac{0.5}{\sqrt{8/3}}\right) \right) + \frac{1}{2} \Phi\left(\frac{-0.5}{\sqrt{8/3}}\right) \xRightarrow{\Phi(x) = 1 - \Phi(-x)}$$

$$P(\text{error}) = 2 \cdot \frac{1}{2} \cdot \left(1 - \Phi(0.306) \right) = 1 - \Phi(0.306) \approx 1 - 0.621 = \underline{\underline{0.379}}$$

(β) Εάν $P(A) = 2P(B)$: $(8) \Rightarrow X^* = 3.5 + \frac{8}{3} \ln 2 \approx 5.35$

$$(9) \xRightarrow[\mu \in \pi \pi \nu]{\text{ομοίως}} P(\text{error}) = \frac{2}{3} (1 - \Phi(1.438)) + \frac{1}{3} \Phi(0.826) \approx$$

$$\approx \frac{2}{3} \cdot 0.075 + \frac{1}{3} \cdot 0.797 = \underline{\underline{0.315}}$$

5. (α) Όπως και στα προηγούμενα προβλήματα: $\begin{cases} \text{Κλάση A} \Rightarrow t=+1 \\ \text{Κλάση B} \Rightarrow t=-1 \end{cases}$

Το training set είναι: $(1,+1), (2,-1), (3,+1), (4,+1), (5,+1), (6,-1)$

Το μοντέλο μας είναι της μορφής: $y = wx + b$

Το σφάλμα είναι

$$F(w,b) = \frac{1}{2} \sum_{i=1}^6 (t_i - (wx_i + b))^2 \quad (10)$$

Για gradient descent έχουμε: $\begin{cases} w_{\text{new}} = w - \eta \frac{\partial F}{\partial w} & (11) \\ b_{\text{new}} = b - \eta \frac{\partial F}{\partial b} & (12) \end{cases}$

Είναι $\frac{\partial F}{\partial w} = - \sum_{i=1}^6 (t_i - (wx_i + b)) x_i \quad (13)$

$\frac{\partial F}{\partial b} = - \sum_{i=1}^6 (t_i - (wx_i + b)) \quad (14)$

Συνεπώς: $\begin{cases} (11) \xrightarrow{(13)} w_{\text{new}} = w + \sum_{i=1}^6 (t_i - (wx_i + b)) x_i = w - \eta \frac{\partial F}{\partial w} \\ (12) \xrightarrow{(14)} b_{\text{new}} = b + \sum_{i=1}^6 (t_i - (wx_i + b)) = b - \eta \frac{\partial F}{\partial b} \end{cases}$

Αρχικοποιούμε το παραπάνω μοντέλο σε $w_0 = 0.1$ και $b_0 = 0.1$

Το η δεν δίνεται, αλλά υποθέτουμε $\eta = 0.01$, για λόγους αντιστάσεως.

Επίσης, ερμηνεύουμε το "3 επαναλήψεις" ως την επεξεργασία όλων των προτύπων από 3 φορές, δηλαδή την ανακύκλωση των βαρών 3 φορές. Θα μπορούσε να γίνει και online ενημέρωση των βαρών: $\begin{cases} w_{\text{new}} = w + \eta (t_i - y_i) x_i \\ w_{\text{old}} = w + \eta (t_i - y_i) \end{cases}$ αλλά κάτι τέτοιο δεν διευκρινίζεται,

όπου τότε 3 επαναλήψεις = επεξεργασία 3 προτύπων.

Συνεπώς, με βάση τους παραπάνω τόνους, θα θεωρήσουμε την batch και όχι την online προσέγγιση που παίρνουμε:

Επανάληψη	w_{old}	b_{old}	$E(w_{old}, b_{old})$	$\partial E / \partial w$	$\partial E / \partial b$	w_{new}	b_{new}	$E(w_{new}, b_{new})$
Επανάληψη 1	0.100	-0.100	3.995	14.20	2.70	-0.042	0.073	2.9059
Επανάληψη 2	-0.042	0.073	2.9059	0.711	-0.444	-0.049	0.077	2.9005
Επανάληψη 3	-0.049	0.077	2.9005	0.157	-0.567	-0.051	0.083	2.8971

Συνεπώς, οι τελικές τιμές των βαρών είναι: $w^{(3)} = -0.051$ και $b^{(3)} = 0.083$
 Επίσης: $E^{(0)} = 3.995 < E^{(1)} = 2.9059 < E^{(2)} = 2.9005 < E^{(3)} = 2.8971$

Παρατηρούμε ότι το σφάλμα μειώνεται πολύ γρήγορα στην 1^η επανάληψη, ενώ έπειτα μειώνεται όλο και πιο αργά, δείχνοντας προς κάποια (θεωρητική) σταθερή τιμή. Αυτή η τιμή είναι μη-μηδενική, καθώς το πρόβλημα είναι μη-γραμμικά διαχωρίσιμο και, συνεπώς, ο LMS, ως διαχωριστικό (discriminative) μοντέλο δεν μπορεί να το διαχωρίσει τέλεια (να βρει το βέλτιστο δηλαδή σημείο). Συνεπώς, το MSE θα μειώνεται μονοτονικά προς μια σταθερή και μη-μηδενική τιμή, λόγω της φύσης του προβλήματος και του διαχωριστικού μας μοντέλου.

Όπως διαεσπρίναμε και προηγουμένως, θεωρήσαμε αυτή την "batch" επεξεργασία των προτύπων. Ωστόσο, για λόγους ληρότητας, παραθέτουμε και την online υλοποίηση (με 3 λήψεις εντός και $3 \times 6 = 18$ αναρνήσεις) με τους κανόνες:

$$\begin{cases} E_i(w, b) = \frac{1}{2} (t_i - y_i)^2 = \frac{1}{2} (t_i - (wx_i + b))^2 \\ \partial E / \partial w = -(t_i - y_i) x_i \Rightarrow w_{new} = w + \eta (t_i - y_i) x_i \\ \partial E / \partial b = -(t_i - y_i) \Rightarrow b_{new} = w + \eta (t_i - y_i) \end{cases}$$

σε παρακάτω πρόγραμμα Python (λόγω της υψηλής υπολογιστικής του δυσκολίας).

analytical_exercises/lms_classification_online.py

```

1 import os
2
3 # -----
4 # Online LMS (Widrow-Hoff) in 1D, 3 epochs
5 # -----
6
7 # Training data: 1D inputs and targets (A -> +1, B -> -1)
8 xs = [1, 2, 3, 4, 5, 6]          # inputs
9 ts = [+1, -1, +1, -1, +1, -1]    # targets
10
11 # Hyperparameters
12 eta = 0.01                       # learning rate
13 epochs = 3                       # number of passes over the dataset
14
15 # Parameter initialization
16 w = 0.1                          # initial weight
17 b = 0.1                          # initial bias
18
19 # Ensure results directory exists
20 os.makedirs("results", exist_ok=True)
21
22 # Output file path
23 output_path = os.path.join("results", "lms_online_log.txt")
24
25 # Open output file (overwrite each run)
26 with open(output_path, "w") as f:
27     # Global header
28     f.write("Online LMS training log (per iteration and per epoch)\n")
29     f.write("=" * 90 + "\n\n")
30
31     # Main training loop over epochs
32     for epoch in range(1, epochs + 1):
33         # -----
34         # 1) Compute INITIAL cost before any updates in this epoch
35         #  $J_{\text{initial}} = (1/2) * \sum_n (t_n - y_n)^2$  with current w, b
36         # -----
37         J_initial = 0.0
38         for x, t in zip(xs, ts):
39             y = w * x + b          # current prediction
40             e = t - y              # current error
41             J_initial += 0.5 * (e ** 2)
42
43         # -----
44         # Print epoch header and per-iteration table header
45         # -----
46         f.write(f"Epoch {epoch}\n")
47         f.write("-" * 90 + "\n")
48         # Column headers for per-iteration table
49         f.write(
50             " i | x | t | w_before | b_before |"
51             " y | e | J_sample | dJ_dw_samp | dJ_db_samp |"
52             " w_after | b_after \n"

```

```

52     )
53     f.write("-" * 90 + "\n")
54
55     # -----
56     # 2) ONLINE UPDATES for this epoch
57     #     One LMS update per sample
58     # -----
59     for i, (x, t) in enumerate(zip(xs, ts), start=1):
60         # Parameters before the update
61         w_before = w
62         b_before = b
63
64         # Forward pass with current parameters
65         y = w_before * x + b_before
66         e = t - y
67
68         # Instantaneous loss for this sample
69         J_sample = 0.5 * (e ** 2)
70
71         # Instantaneous gradients (for the sample loss J_n)
72         # dJ/dw = -(t - y) * x = -e * x
73         # dJ/db = -(t - y)      = -e
74         dJ_dw_sample = -e * x
75         dJ_db_sample = -e
76
77         # Online LMS update (Widrow-Hoff rule)
78         w_after = w_before + eta * e * x
79         b_after = b_before + eta * e
80
81         # Commit the updated parameters
82         w = w_after
83         b = b_after
84
85         # Log this iteration as an aligned table row
86         f.write(
87             f"{i:3d} |"
88             f" {x:3d} |"
89             f" {t:3d} |"
90             f" {w_before:11.6f} |"
91             f" {b_before:11.6f} |"
92             f" {y:11.6f} |"
93             f" {e:11.6f} |"
94             f" {J_sample:11.6f} |"
95             f" {dJ_dw_sample:10.6f} |"
96             f" {dJ_db_sample:10.6f} |"
97             f" {w_after:11.6f} |"
98             f" {b_after:11.6f}\n"
99         )
100
101     # -----
102     # 3) Compute FINAL cost and gradients after updates
103     #     J_final      = (1/2) * sum_n (t_n - y_n)^2
104     #     dJ_dw_last = - sum_n (t_n - y_n) * x_n
105     #     dJ_db_last = - sum_n (t_n - y_n)

```

```

106     # -----
107     J_final = 0.0
108     dJ_dw_final = 0.0
109     dJ_db_final = 0.0
110
111     for x, t in zip(xs, ts):
112         y = w * x + b          # prediction with UPDATED parameters
113         e = t - y
114
115         # Accumulate final cost
116         J_final += 0.5 * (e ** 2)
117
118         # Accumulate final batch gradients at end of epoch
119         dJ_dw_final += -e * x
120         dJ_db_final += -e
121
122     # -----
123     # 4) Epoch-level summary table
124     # -----
125     f.write("-" * 90 + "\n")
126     f.write("Epoch summary:\n")
127     f.write(
128         "    w_final    |    b_final    |    J_initial    |    J_final    |    dJ_dw_final    |
dJ_db_final\n"
129     )
130     f.write(
131         f" {w:11.6f} | {b:11.6f} |"
132         f" {J_initial:11.6f} | {J_final:11.6f} |"
133         f" {dJ_dw_final:11.6f} | {dJ_db_final:11.6f}\n"
134     )
135     f.write("\n\n")
136

```

Online LMS training log (per iteration and per epoch)

Epoch 1

i	x	t	w_before	b_before	y	e
J_sample	dJ_dw_samp	dJ_db_samp	w_after	b_after		
1	1	1	0.100000	0.100000	0.200000	0.800000
0.320000	-0.800000	-0.800000	0.108000	0.108000		
2	2	-1	0.108000	0.108000	0.324000	-1.324000
0.876488	2.648000	1.324000	0.081520	0.094760		
3	3	1	0.081520	0.094760	0.339320	0.660680
0.218249	-1.982040	-0.660680	0.101340	0.101367		
4	4	-1	0.101340	0.101367	0.506728	-1.506728
1.135115	6.026914	1.506728	0.041071	0.086300		
5	5	1	0.041071	0.086300	0.291656	0.708344
0.250876	-3.541721	-0.708344	0.076488	0.093383		
6	6	-1	0.076488	0.093383	0.552314	-1.552314
1.204839	9.313883	1.552314	-0.016650	0.077860		

Epoch summary:

w_final	b_final	J_initial	J_final	dJ_dw_final
dJ_db_final				
-0.016650	0.077860	3.995000	2.953625	3.119874
0.117501				

Epoch 2

i	x	t	w_before	b_before	y	e
J_sample	dJ_dw_samp	dJ_db_samp	w_after	b_after		
1	1	1	-0.016650	0.077860	0.061209	0.938791
0.440664	-0.938791	-0.938791	-0.007262	0.087248		
2	2	-1	-0.007262	0.087248	0.072723	-1.072723
0.575367	2.145446	1.072723	-0.028717	0.076520		
3	3	1	-0.028717	0.076520	-0.009630	1.009630
0.509677	-3.028891	-1.009630	0.001572	0.086617		
4	4	-1	0.001572	0.086617	0.092905	-1.092905
0.597220	4.371619	1.092905	-0.042144	0.075688		
5	5	1	-0.042144	0.075688	-0.135033	1.135033
0.644150	-5.675166	-1.135033	0.014607	0.087038		
6	6	-1	0.014607	0.087038	0.174683	-1.174683
0.689940	7.048097	1.174683	-0.055874	0.075291		

Epoch summary:

w_final	b_final	J_initial	J_final	dJ_dw_final
dJ_db_final				
-0.055874	0.075291	2.953625	2.903087	-0.503373
-0.721596				

Epoch 3

i	x	t	w_before	b_before	y	e
---	---	---	----------	----------	---	---

J_sample	dJ_dw_samp	dJ_db_samp	w_after	b_after

1	1	1	-0.055874	0.075291
0.480771	-0.980582	-0.980582	-0.046068	0.019418
2	2	-1	-0.046068	0.085097
0.492986	1.985923	0.992962	-0.065927	-0.007038
3	3	1	-0.065927	0.075167
0.630130	-3.367840	-1.122613	-0.122613	0.075167
4	4	-1	-0.032249	-0.122613
0.458307	3.829598	0.957400	-0.032249	0.086394
5	5	1	-0.070544	-0.042600
0.813964	-6.379514	-1.275903	-0.070544	-0.957400
6	6	-1	-0.006749	0.076820
0.550287	6.294495	1.049083	-0.006749	-0.275903

Epoch summary:				
w_final	b_final	J_initial	J_final	dJ_dw_final
dJ_db_final				
-0.069694	0.079088	2.903087	2.914937	-1.681338
-0.989054				

2. ΗΜΗ

(α) Ορίζουμε τη μεταβλητή: $\alpha_t(j) = P(O_0, \dots, O_t, q_t = j)$.
Συνεπώς, υπολογίζουμε την πιθανότητα $P(O_0 = H, O_1 = H, O_2 = T)$ διαδοχικά, ως εξής:

Βήμα 1: $t=0$ και $O_0 = H$

$\alpha_0(j) = \pi_j \cdot b_j(H)$, χρησιμοποιώντας τις α priori πιθανότητες

Αντικαθιστώντας:

$$\alpha_0(1) = \frac{1}{3} \cdot 0.5 \approx 0.1667$$

$$\alpha_0(2) = \frac{1}{3} \cdot 0.7 \approx 0.2333$$

$$\alpha_0(3) = \frac{1}{3} \cdot 0.25 \approx 0.0833$$

Συνεπώς: $\alpha_0 = (0.1667, 0.2333, 0.0833)$

Βήμα 2: $t=1$ και $O_1 = H$

Τώρα, για κάθε κατάσταση j είναι: $\alpha_1(j) = \left(\sum_{i=1}^3 \alpha_0(i) a_{ij} \right) b_j(H)$

Επομένως, έχουμε:

- $j=1 \Rightarrow \alpha_1(1) = \left(\sum_{i=1}^3 \alpha_0(i) a_{i1} \right) b_1(H)$

$$\sum_{i=1}^3 \alpha_0(i) a_{i1} = 0.1667 \cdot 0.8 + 0.2333 \cdot 0.2 + 0.0833 \cdot 0.2 \approx 0.1967$$

$$\text{Άρα: } \alpha_1(1) = 0.1967 \cdot 0.5 = 0.0983$$

- $j=2 \Rightarrow \alpha_1(2) = \left(\sum_{i=1}^3 \alpha_0(i) a_{i2} \right) b_2(H)$

$$\sum_{i=1}^3 \alpha_0(i) a_{i2} = 0.1667 \cdot 0.1 + 0.2333 \cdot 0.7 + 0.0833 \cdot 0.1 \approx 0.1883$$

Άρα: $\alpha_1(2) = 0.1883 \cdot 0.7 \approx 0.1318$

• $j=3 \Rightarrow \alpha_1(3) = \left(\sum_{i=1}^3 \alpha_0(i) \alpha_{i3} \right) b_3(H)$

$\sum_{i=1}^3 \alpha_0(i) \alpha_{i3} = 0.1667 \cdot 0.1 + 0.2333 \cdot 0.1 + 0.0833 \cdot 0.7 \approx 0.0983$

Άρα: $\alpha_1(3) = 0.0983 \cdot 0.25 = 0.0246$

Συνεπώς: $\alpha_1 = (0.0983, 0.1318, 0.0246)$

Βήμα 3: $t=2$ και $\alpha_2 = T$

Τώρα, για κάθε κατάσταση j είναι: $\alpha_2(j) = \left(\sum_{i=1}^3 \alpha_1(i) \alpha_{ij} \right) b_j(T)$

Επομένως, έχουμε: • $j=1 \Rightarrow \alpha_2(1) = \left(\sum_{i=1}^3 \alpha_1(i) \alpha_{i1} \right) b_1(T)$

$\sum_{i=1}^3 \alpha_1(i) \alpha_{i1} = 0.0983 \cdot 0.8 + 0.1318 \cdot 0.2 + 0.0246 \cdot 0.2 \approx 0.1099$

$\alpha_2(1) = 0.1099 \cdot 0.5 \approx 0.5050$

• $j=2 \Rightarrow \alpha_2(2) = \left(\sum_{i=1}^3 \alpha_1(i) \alpha_{i2} \right) b_2(T)$

$\sum_{i=1}^3 \alpha_1(i) \alpha_{i2} = 0.0983 \cdot 0.1 + 0.1318 \cdot 0.7 + 0.0246 \cdot 0.1 \approx 0.1046$

$\alpha_2(2) = 0.1046 \cdot 0.3 \approx 0.0314$

• $j=3 \Rightarrow \alpha_2(3) = \left(\sum_{i=1}^3 \alpha_1(i) \alpha_{i3} \right) b_3(T)$

$\sum_{i=1}^3 \alpha_1(i) \alpha_{i3} = 0.0983 \cdot 0.1 + 0.1318 \cdot 0.1 + 0.0246 \cdot 0.7 \approx 0.0402$

$\alpha_2(3) = 0.0402 \cdot 0.75 \approx 0.0302$

Συνεπώς: $\alpha_2 = (0.5050, 0.0314, 0.0302) \leftarrow \begin{cases} \text{F210.0} = \pi_0(1), \delta & L=i \\ \text{F210.0} = \pi_0(2), \delta & L=i \\ \text{F210.0} = \pi_0(3), \delta & L=i \end{cases} \leftarrow \varepsilon=j$

Τελικά: $P(O_0=H, O_1=H, O_2=T) = \sum_{i=1}^3 \alpha_2(i) = 0.5050 + 0.0314 + 0.0302 \approx 0.1165$

Αυτό, είναι και το ζητούμενο: $(\varepsilon, \psi) \leftarrow \begin{cases} \text{F210.0} = \pi_0(1), \delta & L=i \\ \text{F210.0} = \pi_0(2), \delta & L=i \\ \text{F210.0} = \pi_0(3), \delta & L=i \end{cases} \leftarrow \varepsilon=j$

(b) Θέλουμε να βρούμε τα $q = (q_1, q_2, q_3)$ με $\arg \max_{q_0, q_1, q_2} P(q_0, q_1, q_2, O_0=H, O_1=H, O_2=T)$.

Ορίζουμε: $\delta_t(j) = \max_{q_0, \dots, q_t=j} P(q_0, \dots, q_t, O_0, \dots, O_t) \leftarrow \begin{cases} \text{F210.0} = \pi_0(1), \delta & L=i \\ \text{F210.0} = \pi_0(2), \delta & L=i \\ \text{F210.0} = \pi_0(3), \delta & L=i \end{cases} \leftarrow \varepsilon=j$

$\psi_t(j) = \arg \max$ κατάσταση $(t-1)$ που μεγιστοποιεί το $\delta_t(j)$ (backpointer)

Συνεπώς, έχουμε τα 2 ακόλουθα βήματα: $(H), \delta, \pi_0(i), \delta \leftarrow \begin{cases} \text{F210.0} = \pi_0(1), \delta & L=i \\ \text{F210.0} = \pi_0(2), \delta & L=i \\ \text{F210.0} = \pi_0(3), \delta & L=i \end{cases} \leftarrow \varepsilon=j$

• Βήμα 1: Αρχικοποίηση ($t=0$) $\leftarrow \begin{cases} \text{F210.0} = \pi_0(1), \delta & L=i \\ \text{F210.0} = \pi_0(2), \delta & L=i \\ \text{F210.0} = \pi_0(3), \delta & L=i \end{cases} \leftarrow \varepsilon=j$

$\delta_0(j) = \pi_j b_j(H)$ και $\psi_0(j) = \text{κενό}$

Είναι: $\delta_0(1) = \pi_1 b_1(H) = 0.1667$
 $\delta_0(2) = \pi_2 b_2(H) = 0.2333$
 $\delta_0(3) = \pi_3 b_3(H) = 0.0667$ $\leftarrow \begin{cases} \text{F210.0} = \pi_0(1), \delta & L=i \\ \text{F210.0} = \pi_0(2), \delta & L=i \\ \text{F210.0} = \pi_0(3), \delta & L=i \end{cases} \leftarrow \varepsilon=j$ όπως και πριν

• Βήμα 2: Δεύτερη παρατήρηση H ($t=1$) $\leftarrow \begin{cases} \text{F210.0} = \pi_0(1), \delta & L=i \\ \text{F210.0} = \pi_0(2), \delta & L=i \\ \text{F210.0} = \pi_0(3), \delta & L=i \end{cases} \leftarrow \varepsilon=j$

$\delta_1(j) = \max_i \delta_0(i) a_{ij} b_j(H)$ και $\psi_1(j) = \arg \max_i \delta_0(i) a_{ij}$

• $j=1 \Rightarrow \begin{cases} i=1 : \delta_0(1) a_{11} = 0.1333 \\ i=2 : \delta_0(2) a_{21} = 0.0467 \\ i=3 : \delta_0(3) a_{31} = 0.0167 \end{cases} \Rightarrow \begin{cases} \psi_1(1) = 1 + \text{F210.0} = \pi_0(1), \delta & L=i \\ \delta_1(1) = 0.1333 b_1(H) = 0.0667 \end{cases} \leftarrow \varepsilon=j$

$$\bullet j=2 \Rightarrow \begin{cases} i=1: \delta_0(1)\alpha_{12} = 0.0167 \\ i=2: \delta_0(2)\alpha_{22} = 0.1633 \\ i=3: \delta_0(3)\alpha_{32} = 0.0083 \end{cases} \Rightarrow \begin{cases} \psi_1(2) = 2 \\ \delta_1(2) = 0.1633 \cdot 0.7 = 0.1143 \end{cases}$$

$$\bullet j=3 \Rightarrow \begin{cases} i=1: \delta_0(1)\alpha_{13} = 0.0167 \\ i=2: \delta_0(2)\alpha_{23} = 0.0233 \\ i=3: \delta_0(3)\alpha_{33} = 0.0583 \end{cases} \Rightarrow \begin{cases} \psi_1(3) = 3 \\ \delta_1(3) = 0.0583 \cdot 0.25 = 0.0146 \end{cases}$$

Jurenius: $\delta_1 = (0.0667, 0.1143, 0.0146)$ και $\psi_1 = (1, 2, 3)$

Βήμα 3: Παρατήρηση T ($t=2$)

$$\delta_2(j) = \max_i \{ \delta_1(i)\alpha_{ij} \} b_j(T) \quad \text{και} \quad \psi_2(j) = \arg \max_i \{ \delta_1(i)\alpha_{ij} \}$$

$$\bullet j=1 \Rightarrow \begin{cases} i=1: \delta_1(1)\alpha_{11} = 0.0533 \\ i=2: \delta_1(2)\alpha_{21} = 0.0229 \\ i=3: \delta_1(3)\alpha_{31} = 0.0029 \end{cases} \Rightarrow \begin{cases} \psi_2(1) = 1 \\ \delta_2(1) = 0.0533 \cdot 0.5 = 0.0267 \end{cases}$$

$$\bullet j=2 \Rightarrow \begin{cases} i=1: \delta_1(1)\alpha_{12} = 0.0067 \\ i=2: \delta_1(2)\alpha_{22} = 0.0800 \\ i=3: \delta_1(3)\alpha_{32} = 0.0015 \end{cases} \Rightarrow \begin{cases} \psi_2(2) = 2 \\ \delta_2(2) = 0.0800 \cdot 0.3 = 0.0240 \end{cases}$$

$$\bullet j=3 \Rightarrow \begin{cases} i=1: \delta_1(1)\alpha_{13} = 0.0067 \\ i=2: \delta_1(2)\alpha_{23} = 0.0114 \\ i=3: \delta_1(3)\alpha_{33} = 0.0102 \end{cases} \Rightarrow \begin{cases} \psi_2(3) = 2 \\ \delta_2(3) = 0.0114 \cdot 0.75 = 0.0086 \end{cases}$$

Jurenius: $\delta_2 = (0.0267, 0.0240, 0.0086)$ και $\psi_2 = (1, 2, 2)$

Επομένως, είναι: $\max_j \delta_2(j) = \delta_2(1) = 0.0267$ και $\psi_2(1) = 1$.

Εν συνεχεία, κάνοντας backtracking προκύπτει:
$$\begin{cases} t=2: \hat{q}_2 = 1 \\ t=1: \hat{q}_1 = \psi_2(\hat{q}_2) = \psi_2(1) = 1 \\ t=0: \hat{q}_0 = \psi_1(\hat{q}_1) = \psi_1(1) = 1 \end{cases}$$

Τελικά, τα ζητούμενα (η πιο πιθανή σειρά καταστάσεων) είναι η:

$$\hat{q} = (\hat{q}_0, \hat{q}_1, \hat{q}_2) = (1, 1, 1)$$

(c) Το μοντέλο μας, με βάση τον ψεύδο-ΕΜ αλγόριθμο (Viterbi Training) εκπαιδεύεται ως εξής:

• E-step: Βρίσκει την πιο πιθανή ακολουθία: $\hat{q}_{0:T-1} = \arg\max_{q_{0:T-1}} P(q_{0:T-1}, O_{0:T-1} | \theta^{(old)})$

Είναι:
$$\begin{cases} \delta_t(j) = \max_{d_0, \dots, d_{t-1}} P(d_0, \dots, d_{t-1}, O_{0:T-1} | \theta^{(old)}) \\ \psi_t(j) = \arg\max_i \{ \delta_{t-1}(i) a_{ij} \} \end{cases}$$

Αρχικοποίηση:
$$\begin{cases} \delta_0(j) = \pi_j^{(old)} b_j^{(old)}(O_0) \\ \psi_0(j) = \text{κενό} \end{cases}$$

Αναδρομή:
$$\begin{cases} \delta_t(j) = \left(\max_i \delta_{t-1}(i) a_{ij}^{(old)} \right) b_j^{(old)}(O_t) \\ \psi_t(j) = \arg\max_i \delta_{t-1}(i) a_{ij}^{(old)} \end{cases}$$

Backtracking:
$$\begin{cases} \hat{q}_{T-1} = \arg\max_j \delta_{T-1}(j) \\ \hat{q}_t = \psi_{t+1}(\hat{q}_{t+1}), \quad t = 0, \dots, T-2 \end{cases}$$

- M-step: Μεγιστοποίηση της εκτίμησης των παραμέτρων $\theta = (\pi, A, B)$ με τη συνάρτηση μέγιστης πιθανοφάνειας:

$$P(\hat{q}_{0:T-1}, O_{0:T-1} | \theta) = \pi_{\hat{q}_0} \prod_{t=1}^{T-1} a_{\hat{q}_{t-1}, \hat{q}_t} \prod_{t=0}^{T-1} b_{\hat{q}_t}(O_t)$$

ή με log-likelihood:

$$\log(P(\hat{q}_{0:T-1}, O_{0:T-1} | \theta)) = \log \pi_{\hat{q}_0} + \sum_{t=1}^{T-1} \log a_{\hat{q}_{t-1}, \hat{q}_t} + \sum_{t=0}^{T-1} \log b_{\hat{q}_t}(O_t)$$

Ισχύουν οι περιορισμοί :

$$\begin{cases} \sum_i \pi_i = 1, \pi_i \geq 0 \\ \sum_j a_{ij} = 1, a_{ij} \geq 0 \\ \sum_b b_j(b) = 1, b_j(b) \geq 0 \end{cases}$$

Καθώς η παραπάνω διαδικασία είναι αρκετά πολυπλοκή (ειδικά για 15 παρατηρήσεις), συντάξαμε το ακόλουθο πρόγραμμα Python, που εκτελεί αριθμώς την παραπάνω λειτουργία και φέρνει μετὰ από το υποπρώτο (d).

(d) Ξεκινώντας από forward-backward το E-step είναι "μολδαίο": υπολογίζουμε κατανομές $\gamma_t(i)$ και $\xi_t(i, j)$, δηλαδή πιθανότητες για το σε ποια κατάσταση βρισκόμαστε και ποιες μεταβάσεις γίνονται σε κάθε χρονική στιγμή, λαμβάνοντας υπόψη όλες τις δυνατές ακολουθίες καταστάσεων. Είναι ένας κλασικός ΕΜ που μεγιστοποιεί την $P(O|\theta)$ και εγγυάται ότι η πιθανότητα των δεδομένων δεν μειώνεται σε κάθε επανάληψη.

Αντίθετα, στο Viterbi-training, το E-step είναι "ακέραιο": βρίσκουμε τη μοναδική πιο πιθανή ακολουθία καταστάσεων και θεωρούμε αυτή ως γνωστή στην εκτίμηση των παραμέτρων. Έτσι, αγνοείται η αβεβαιότητα στις κρυφές καταστάσεις και ουσιαστικά κάνουμε ΕΜ πάνω σε 1 μόνο διαδρομή, κάτι το οποίο δεν μεγιστοποιεί απαραίτητα την πραγματική $P(O|\theta)$.

Έτσι, ο F-B είναι θεωρητικά και πρακτικά καλύτερος στην εκτίμηση των παραμέτρων και τη γενίκευση και είναι πιο σιωδρός και ομαλός. Ο Viterbi-training είναι λιγότερο αξιόπιστος και πιο "χοιροειδής" και πιο ευαίσθητος σε τοπικά ακρότατα και μπορεί να οδηγήσει σε καταρρευση του μοντέλου σε μια μόνο κερπάρχη κατάσταση, αλλά είναι σημαντικά πιο γρήγορος.

analytical_exercises/hmm_viterbi_training.py

```

1  #!/usr/bin/env python3
2  """
3  Minimal Viterbi-training script for Problem 2(c).
4
5  What it does:
6  1. Defines the HMM of Problem 2:
7     - States: 1, 2, 3
8     - Observations: H, T
9     - Given initial pi, A, B.
10  2. Uses the training sequence:
11     HHTTTHHTTTTTHHHH
12  3. Runs several epochs of Viterbi-training:
13     (E-step: Viterbi, M-step: frequency re-estimation).
14  4. After EACH epoch, logs:
15     - best Viterbi path
16     - joint probability of path and observations
17     - updated pi, A, B
18  5. Writes everything into ONE file:
19     results/hmm_viterbi_training_log.txt
20  """
21
22  import os
23
24
25  def viterbi(pi, A, B, obs_seq, obs_map, state_labels):
26      """
27      Run the Viterbi algorithm for a single sequence.
28
29      Parameters
30      -----
31      pi : list[float]
32          Initial distribution, length N.
33      A : list[list[float]]
34          Transition matrix  $A[i][j] = P(q_t = j \mid q_{t-1} = i)$ .
35          Shape N x N.
36      B : list[list[float]]
37          Emission matrix  $B[j][k] = P(O_t = obs_k \mid q_t = j)$ .
38          Shape N x M, where M = number of observation symbols.
39      obs_seq : list[str]
40          Observation sequence (e.g. ['H','H','T',...]).
41      obs_map : dict[str, int]
42          Maps each observation symbol to a column index in B.
43      state_labels : list[int]
44          Human-readable labels for states (e.g. [1,2,3]).
45
46      Returns
47      -----
48      best_path_labels : list[int]
49          Most likely state sequence (MAP path) in terms of state labels.
50      best_prob : float
51          Joint probability of that path with the observations.

```

```

52     """
53     T = len(obs_seq)          # length of observation sequence
54     N = len(state_labels)     # number of states
55
56     # delta[t][j] = max probability of any path ending in state j at time t
57     # psi[t][j] = argmax previous state for that path (backpointer)
58     delta = [[0.0] * N for _ in range(T)]
59     psi = [[-1] * N for _ in range(T)]
60
61     # ----- Initialization (t = 0) -----
62     o0 = obs_map[obs_seq[0]]
63     for j in range(N):
64         delta[0][j] = pi[j] * B[j][o0]
65         psi[0][j] = -1 # no predecessor at t = 0
66
67     # ----- Recursion (t = 1, ..., T-1) -----
68     for t in range(1, T):
69         ot = obs_map[obs_seq[t]]
70         for j in range(N):
71             # For each destination state j, find best previous state i
72             best_val = -1.0
73             best_i = -1
74             for i in range(N):
75                 val = delta[t - 1][i] * A[i][j]
76                 if val > best_val:
77                     best_val = val
78                     best_i = i
79             # Multiply by emission probability for observation at time t
80             delta[t][j] = best_val * B[j][ot]
81             psi[t][j] = best_i
82
83     # ----- Termination -----
84     last_t = T - 1
85     best_last_state = max(range(N), key=lambda j: delta[last_t][j])
86     best_prob = delta[last_t][best_last_state]
87
88     # ----- Backtracking -----
89     best_path_indices = [best_last_state] * T
90     for t in range(last_t, 0, -1):
91         best_path_indices[t - 1] = psi[t][best_path_indices[t]]
92
93     # Convert internal indices (0..N-1) to external labels (1..N)
94     best_path_labels = [state_labels[idx] for idx in best_path_indices]
95     return best_path_labels, best_prob
96
97
98 def reestimate_parameters(path_labels, obs_seq, obs_map,
99                           num_states, num_obs,
100                           pi_old, A_old, B_old):
101     """
102     One M-step of Viterbi-training (hard EM).
103
104     Given:
105     - path_labels: Viterbi-decoded state sequence (labels like 1,2,3),

```

```

106     - obs_seq: observation sequence (e.g. ['H','T',...]),
107     treat them as fully observed and compute ML estimates:
108
109     pi_new[i] ~ frequency of starting in state i
110     A_new[i][j] ~ frequency of i->j transitions (normalized over j)
111     B_new[j][k] ~ frequency of symbol k in state j (normalized over k)
112
113     If a state never appears in the path (no counts),
114     we fall back to the OLD parameters for that state
115     to avoid degenerate rows.
116
117     Parameters
118     -----
119     path_labels : list[int]
120         State labels (1..num_states) for each time step.
121     obs_seq : list[str]
122         Observation sequence.
123     obs_map : dict[str, int]
124         Maps observation symbol -> index.
125     num_states : int
126         Number of states.
127     num_obs : int
128         Number of distinct observation symbols.
129     pi_old, A_old, B_old :
130         Old parameters (used as fallback if there are no counts).
131
132     Returns
133     -----
134     pi_new : list[float]
135     A_new : list[list[float]]
136     B_new : list[list[float]]
137     """
138     T = len(obs_seq)
139
140     # ----- Initial distribution pi -----
141     pi_new = [0.0] * num_states
142     start_state_idx = path_labels[0] - 1 # convert label (1..N) to index (0..N-1)
143     pi_new[start_state_idx] = 1.0
144
145     # ----- Transition counts -----
146     trans_counts = [[0] * num_states for _ in range(num_states)]
147     for t in range(1, T):
148         i = path_labels[t - 1] - 1 # from state index
149         j = path_labels[t] - 1 # to state index
150         trans_counts[i][j] += 1
151
152     # Normalize counts -> A_new.
153     # If a state i never appears (row_sum == 0), keep A_old[i].
154     A_new = [[0.0] * num_states for _ in range(num_states)]
155     for i in range(num_states):
156         row_sum = sum(trans_counts[i])
157         if row_sum > 0:
158             A_new[i] = [c / row_sum for c in trans_counts[i]]
159         else:

```

```

160         # No transitions from state i observed: keep old row.
161         A_new[i] = A_old[i][:]
162
163     # ----- Emission counts -----
164     emit_counts = [[0] * num_obs for _ in range(num_states)]
165     state_visit_counts = [0] * num_states
166     for t, label in enumerate(path_labels):
167         s_idx = label - 1          # state index
168         o_idx = obs_map[obs_seq[t]] # observation index
169         emit_counts[s_idx][o_idx] += 1
170         state_visit_counts[s_idx] += 1
171
172     # Normalize counts -> B_new.
173     # If a state j is never visited, keep B_old[j].
174     B_new = [[0.0] * num_obs for _ in range(num_states)]
175     for j in range(num_states):
176         total = state_visit_counts[j]
177         if total > 0:
178             B_new[j] = [c / total for c in emit_counts[j]]
179         else:
180             B_new[j] = B_old[j][:]
181
182     return pi_new, A_new, B_new
183
184
185 def log_params(f, header, pi, A, B):
186     """
187     Helper function to write parameters nicely into the log file.
188
189     Parameters
190     -----
191     f : file object
192         Open text file for writing.
193     header : str
194         A small header/title to print before the parameters.
195     pi : list[float]
196         Initial distribution.
197     A : list[list[float]]
198         Transition matrix.
199     B : list[list[float]]
200         Emission matrix (rows = states, cols = [H, T]).
201     """
202     f.write(header + "\n")
203     f.write("pi:\n")
204     f.write(" " + " ".join(f"{x:.6f}" for x in pi) + "\n\n")
205
206     f.write("A (rows = from state, cols = to state):\n")
207     for i, row in enumerate(A, start=1):
208         row_str = " ".join(f"{x:.6f}" for x in row)
209         f.write(f" from state {i}: {row_str}\n")
210     f.write("\n")
211
212     f.write("B (rows = states 1..3, cols = [H, T]):\n")
213     for j, row in enumerate(B, start=1):

```

```

214         row_str = " ".join(f"{x:.6f}" for x in row)
215         f.write(f" state {j}: {row_str}\n")
216     f.write("\n")
217
218
219 def main():
220     # ----- Basic settings -----
221     NUM_EPOCHS = 5 # number of Viterbi-training epochs (change if you want)
222
223     # Ensure output directory exists
224     os.makedirs("results", exist_ok=True)
225     log_path = os.path.join("results", "hmm_viterbi_training_log.txt")
226
227     # ----- HMM definition (Problem 2) -----
228     # States are labeled as 1, 2, 3
229     state_labels = [1, 2, 3]
230
231     # Observations: 'H' and 'T'
232     obs_symbols = ['H', 'T']
233     obs_map = {'H': 0, 'T': 1}
234
235     num_states = len(state_labels)
236     num_obs = len(obs_symbols)
237
238     # Initial distribution pi
239     pi = [1.0 / 3.0, 1.0 / 3.0, 1.0 / 3.0]
240
241     # Transition matrix A (rows = from-state, cols = to-state)
242     A = [
243         [0.8, 0.1, 0.1],
244         [0.2, 0.7, 0.1],
245         [0.2, 0.1, 0.7],
246     ]
247
248     # Emission matrix B (rows = states 1..3, cols = [H, T])
249     B = [
250         [0.5, 0.5], # state 1: P(H)=0.5, P(T)=0.5
251         [0.7, 0.3], # state 2: P(H)=0.7, P(T)=0.3
252         [0.25, 0.75], # state 3: P(H)=0.25, P(T)=0.75
253     ]
254
255     # Training observation sequence from Question (c)
256     obs_seq = list("HHTTTHHTTTTHHHH")
257
258     # ----- Open log file and write header -----
259     with open(log_path, "w", encoding="utf-8") as f:
260         f.write("Viterbi-training for HMM (Problem 2(c))\n")
261         f.write("=====\n\n")
262         f.write(f"Observation sequence: {''.join(obs_seq)}\n")
263         f.write(f"Number of epochs: {NUM_EPOCHS}\n\n")
264
265         # Log initial parameters (epoch 0)
266         log_params(f, header="Initial parameters (epoch 0):", pi=pi, A=A, B=B)
267

```

```
268 # ----- Viterbi-training epochs -----
269 for epoch in range(1, NUM_EPOCHS + 1):
270     # E-step: Viterbi decoding with current parameters
271     best_path, best_prob = viterbi(pi, A, B, obs_seq, obs_map, state_labels)
272
273     # M-step: update parameters using decoded path
274     pi_new, A_new, B_new = reestimate_parameters(
275         path_labels=best_path,
276         obs_seq=obs_seq,
277         obs_map=obs_map,
278         num_states=num_states,
279         num_obs=num_obs,
280         pi_old=pi,
281         A_old=A,
282         B_old=B,
283     )
284
285     # Overwrite parameters with updated ones
286     pi, A, B = pi_new, A_new, B_new
287
288     # Log results for this epoch
289     f.write(f"Epoch {epoch}\n")
290     f.write("-----\n")
291     f.write(f"Best Viterbi path (states): {best_path}\n")
292     f.write(f"Joint probability of best path: {best_prob:.10e}\n\n")
293
294     log_params(
295         f,
296         header=f"Parameters after epoch {epoch}:",
297         pi=pi,
298         A=A,
299         B=B,
300     )
301
302     # ----- Final summary -----
303     f.write("Final parameters after all epochs:\n")
304     f.write("-----\n")
305     log_params(f, header="Final (same as last epoch):", pi=pi, A=A, B=B)
306
307     # The log file now contains:
308     # - initial parameters
309     # - per-epoch path and probability
310     # - per-epoch parameters
311     # - final parameters
312
313
314 if __name__ == "__main__":
315     main()
316
```

Viterbi-training for HMM (Problem 2(c))

=====

Observation sequence: HHTTTHHTTTTTHHHH
Number of epochs: 5

Initial parameters (epoch 0):

π :
0.333333 0.333333 0.333333

A (rows = from state, cols = to state):
from state 1: 0.800000 0.100000 0.100000
from state 2: 0.200000 0.700000 0.100000
from state 3: 0.200000 0.100000 0.700000

B (rows = states 1..3, cols = [H, T]):
state 1: 0.500000 0.500000
state 2: 0.700000 0.300000
state 3: 0.250000 0.750000

Epoch 1

Best Viterbi path (states): [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Joint probability of best path: 1.7895697067e-07

Parameters after epoch 1:

π :
1.000000 0.000000 0.000000

A (rows = from state, cols = to state):
from state 1: 1.000000 0.000000 0.000000
from state 2: 0.200000 0.700000 0.100000
from state 3: 0.200000 0.100000 0.700000

B (rows = states 1..3, cols = [H, T]):
state 1: 0.500000 0.500000
state 2: 0.700000 0.300000
state 3: 0.250000 0.750000

Epoch 2

Best Viterbi path (states): [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Joint probability of best path: 1.5258789062e-05

Parameters after epoch 2:

π :
1.000000 0.000000 0.000000

A (rows = from state, cols = to state):
from state 1: 1.000000 0.000000 0.000000
from state 2: 0.200000 0.700000 0.100000
from state 3: 0.200000 0.100000 0.700000

B (rows = states 1..3, cols = [H, T]):
state 1: 0.500000 0.500000
state 2: 0.700000 0.300000
state 3: 0.250000 0.750000

Epoch 3

Best Viterbi path (states): [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Joint probability of best path: 1.5258789062e-05

Parameters after epoch 3:

```
pi:
  1.000000 0.000000 0.000000
```

```
A (rows = from state, cols = to state):
  from state 1: 1.000000 0.000000 0.000000
  from state 2: 0.200000 0.700000 0.100000
  from state 3: 0.200000 0.100000 0.700000
```

```
B (rows = states 1..3, cols = [H, T]):
  state 1: 0.500000 0.500000
  state 2: 0.700000 0.300000
  state 3: 0.250000 0.750000
```

Epoch 4

```
-----
Best Viterbi path (states): [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Joint probability of best path: 1.5258789062e-05
```

Parameters after epoch 4:

```
pi:
  1.000000 0.000000 0.000000
```

```
A (rows = from state, cols = to state):
  from state 1: 1.000000 0.000000 0.000000
  from state 2: 0.200000 0.700000 0.100000
  from state 3: 0.200000 0.100000 0.700000
```

```
B (rows = states 1..3, cols = [H, T]):
  state 1: 0.500000 0.500000
  state 2: 0.700000 0.300000
  state 3: 0.250000 0.750000
```

Epoch 5

```
-----
Best Viterbi path (states): [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Joint probability of best path: 1.5258789062e-05
```

Parameters after epoch 5:

```
pi:
  1.000000 0.000000 0.000000
```

```
A (rows = from state, cols = to state):
  from state 1: 1.000000 0.000000 0.000000
  from state 2: 0.200000 0.700000 0.100000
  from state 3: 0.200000 0.100000 0.700000
```

```
B (rows = states 1..3, cols = [H, T]):
  state 1: 0.500000 0.500000
  state 2: 0.700000 0.300000
  state 3: 0.250000 0.750000
```

Final parameters after all epochs:

```
-----
Final (same as last epoch):
```

```
pi:
  1.000000 0.000000 0.000000
```

```
A (rows = from state, cols = to state):
  from state 1: 1.000000 0.000000 0.000000
  from state 2: 0.200000 0.700000 0.100000
  from state 3: 0.200000 0.100000 0.700000
```

```
B (rows = states 1..3, cols = [H, T]):
  state 1: 0.500000 0.500000
```

```
state 2: 0.700000 0.300000
state 3: 0.250000 0.750000
```

3. Backpropagation

Σύμφωνα με το δοσμένο νευρωνικό δίκτυο έχουμε τις ακόλουθες αναπαρά-
στάσεις για τις διάφορες καταστάσεις που σημειώνονται:

$$S_1 = w_1 x_1 + w_2 x_2 = -1.7x_1 + 0.1x_2 \quad (1)$$

$$S_2 = w_3 x_3 + w_4 x_4 = -0.6x_3 - 1.8x_4 \quad (2)$$

$$h_1 = \sigma(s_1) = \frac{1}{1 + e^{-s_1}} = \frac{1}{1 + \exp(-1.7x_1 + 0.1x_2)} \quad (3)$$

$$h_2 = \sigma(s_2) = \frac{1}{1 + e^{-s_2}} = \frac{1}{1 + \exp(-0.6x_3 - 1.8x_4)} \quad (4)$$

$$S_3 = w_5 h_1 + w_6 h_2 = -0.2h_1 + 0.5h_2 \quad (5)$$

$$\hat{y} = \sigma(s_3) = \frac{1}{1 + e^{-s_3}} = \frac{1}{1 + \exp(-0.2h_1 + 0.5h_2)} \quad (6)$$

Επίσης, θέλουμε να υπολογίσουμε το $\frac{\partial L}{\partial w_1}$, που με τον κανόνα της αλυσίδας,
γράφεται ως:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial s_3} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial s_3} \cdot \frac{\partial s_3}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1} \iff$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial s_3} \cdot \frac{\partial s_3}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1} \quad (7)$$

Χρησιμοποιώντας τις σχέσεις (1)-(6) υπολογίζουμε καλύτερα από τους όρους της (7). Όμως, πρώτα υπολογίζουμε την τιμή της εφόδου $\hat{y}$ και $(x_1, x_2, x_3, x_4) = (-0.7, 1.2, 1.1, -2)$:

$$\begin{aligned} (1) \Rightarrow S_1 &= 1.19 + 0.12 = 1.31 \\ (2) \Rightarrow S_2 &= -0.66 + 3.6 = 2.94 \end{aligned} \quad \left. \begin{array}{l} (3) \\ (4) \end{array} \right\} \begin{aligned} h_1 &= \frac{1}{1+e^{-1.31}} = 0.788 \text{ και } h_2 = \frac{1}{1+e^{-2.94}} = 0.950 \end{aligned}$$

$$\text{Άρα: } (5) \Rightarrow S_3 = -0.1576 + 0.475 = 0.3174 \xrightarrow{(6)} \hat{y} = \frac{1}{1+e^{-0.3174}} = 0.579$$

$$\text{Συνεπώς: } \cdot \frac{\partial L}{\partial \hat{y}} = 2 \|y - \hat{y}\| = 2 \cdot \|0.5 - 0.579\| = 0.158$$

$$\cdot \frac{\partial \hat{y}}{\partial S_3} \stackrel{(6)}{=} \frac{e^{-S_3}}{(1+e^{-S_3})^2} = 0.230$$

$$\cdot \frac{\partial S_3}{\partial h_1} \stackrel{(5)}{=} -0.2$$

$$\cdot \frac{\partial h_1}{\partial S_1} \stackrel{(3)}{=} \frac{e^{-S_1}}{(1+e^{-S_1})^2} = 0.167$$

$$\cdot \frac{\partial S_1}{\partial w_1} \stackrel{(1)}{=} -0.7$$

$$\text{Συνεπώς: } \frac{\partial L}{\partial w_1} \stackrel{(7)}{=} 0.158 \cdot 0.230 \cdot (-0.2) \cdot 0.167 \cdot (-0.7) = 0.00084963$$

το οποίο είναι και το ζητούμενο (με προσχηματισμούς ανά βήμα, όλα σχετικά κατά περίεργα).

$$(7) \quad \frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial S_3} \cdot \frac{\partial S_3}{\partial h_1} \cdot \frac{\partial h_1}{\partial S_1} \cdot \frac{\partial S_1}{\partial w_1}$$

4. Expectation - Maximization

(α) Έχουμε ότι $\theta^{(0)} = (\theta_1, \theta_2) = (2, 3)$ και θα υπολογίσουμε το E-step του αλγορίθμου EM για το $Q(\theta, \theta^{(0)})$.

Σύμφωνα με τα δεδομένα έχουμε ότι :

$$D = D_{\text{good}} \cup D_{\text{bad}} = \{x_1, x_2, x_3\} = \{(x_{11}, x_{12}), (x_{21}, x_{22}), (x_{31}, x_{32})\} = \{(1, 2), (4, 5), (2, x_{32})\}$$

Επίσης : $D_{\text{bad}} = x_{32}$ και $D_{\text{good}} = D - D_{\text{bad}}$.

Συνεπώς, έχουμε ότι :

$$Q(\theta, \theta^{(0)}) = E_{D_{\text{bad}}} \{ \ln p(D | \theta^{(0)}; D_{\text{good}}) \} = E_{x_{32}} \{ \ln p(D | \theta^{(0)}; D_{\text{good}}) \} =$$

$$= \int_{-\infty}^{+\infty} \left(\sum_{k=1}^2 \ln p(x_k | \theta) \right) p(x_{32} | \theta^{(0)}, x_{31}) dx_{32} =$$

$$= \sum_{k=1}^2 \ln p(x_k | \theta) + \int_{-\infty}^{+\infty} \ln p(x_3 | \theta) p(x_{32} | \theta^{(0)}, x_{31}) dx_{32} \quad (1)$$

$$\text{Όπως } p(x_k | \theta) = p(x_{k1}, x_{k2} | \theta) = \frac{1}{\theta_1 \theta_2} e^{-x_{k1}/\theta_1}, \quad x_{k1} \geq 0 \text{ και } 0 \leq x_{k2} \leq \theta_2 \quad (2)$$

$$\text{Επομένως : (1)} \xRightarrow{(2)} Q(\theta, \theta^{(0)}) \stackrel{x_{11}=1}{\stackrel{x_{21}=4}} = \ln\left(\frac{1}{\theta_1 \theta_2}\right) - \frac{1}{\theta_1} + \ln\left(\frac{1}{\theta_1 \theta_2}\right) - \frac{4}{\theta_1} + \int_0^3 \ln\left(\frac{1}{\theta_1 \theta_2} e^{-2\theta_1}\right) \cdot \frac{1}{3} \cdot dx_{32} =$$

$$= -2\ln\theta_1 - 2\ln\theta_2 - \frac{5}{\theta_1} + \ln\left(\frac{1}{\theta_1 \theta_2} e^{-2/\theta_1}\right) \cdot \frac{1}{3} \cdot [x_{32}]_0^3 \Rightarrow$$

$$\Rightarrow \boxed{Q(\theta, \theta^{(0)}) = -3\ln\theta_1 - 3\ln\theta_2 - \frac{7}{\theta_1}} \quad (3)$$

Η παραπάνω έκφραση αποτελεί το E-step του αλγορίθμου EM.

Σημαντική σημείωση : Η $p(x_i) = \frac{1}{\theta_1} e^{-x_i/\theta_1}$ της εκτίμησης δεν κανονικοποιείται και δεν αποτελεί σ.π.π., ούτε και για την τιμή $\theta_1^{(0)} = 2$. Συνεπώς, θεωρώντας το τυπογραφικό, λύσαμε την άσκηση με $p(x_i) = \frac{1}{\theta_1} e^{-x_i/\theta_1}$ που κανονικοποιείται.

(b) Τώρα, θα προχωρήσουμε στο M-step του αλγορίθμου EM.
 Δηλαδή, θα βρούμε το $\theta = (\theta_1, \theta_2)$, το οποίο μεγιστοποιεί το $Q(\theta, \theta^{(0)})$ του προηγούμενου βήματος, με τη μέθοδο της μέγιστης πιθανοφάνειας.
 Επομένως:

$$\frac{\partial Q(\theta, \theta^{(0)})}{\partial \theta_1} = 0 \stackrel{(3)}{\Leftrightarrow} -\frac{3}{\theta_1} + \frac{7}{\theta_1^2} = 0 \Leftrightarrow 3\theta_1 - 7 = 0 \Leftrightarrow \boxed{\theta_1 = 7/3}$$

$$\frac{\partial Q(\theta, \theta^{(0)})}{\partial \theta_2} = 0 \stackrel{(4)}{\Leftrightarrow} -\frac{3}{\theta_2} = 0$$

Η παραπάνω εξίσωση είναι αδύνατη, οπότε φθινούσα συνάρτηση του θ_2 .

Έχουμε τον περιορισμό (από την $p(x_2)$ της εκτίμησης) : $0 \leq x_2 \leq \theta_2$

Συνεπώς : $\theta_2 \geq x_{k2}, \forall k \in \{1, 2, 3\}$

Είναι $x_{k2, \max} = x_{22} = 5$.

Άρα $\theta_2 \geq 5$ και αφού $\frac{\partial Q}{\partial \theta_2} < 0 \Rightarrow \boxed{\theta_2 = 5}$

Συνεπώς : $\boxed{\theta = \theta^{(1)} = \left(\frac{3}{7}, 5 \right)}$ οι ζητούμενες τιμές των παραμέτρων.

(c) Για $\theta^{(0)}$:

$$p(x_1) \sim \begin{cases} \frac{1}{2} e^{-x_1/2}, & x_1 \geq 0 \\ 0, & \text{αλλιώς} \end{cases} \quad p(x_2) \sim \begin{cases} \frac{1}{3}, & 0 \leq x_2 \leq 3 \\ 0, & \text{αλλιώς} \end{cases}$$

Για $\theta^{(1)}$:

$$p(x_1) \sim \begin{cases} \frac{7}{3} e^{-\frac{3}{7}x_1}, & x_1 \geq 0 \\ 0, & \text{αλλιώς} \end{cases} \quad p(x_2) \sim \begin{cases} \frac{1}{5}, & 0 \leq x_2 \leq 5 \\ 0, & \text{αλλιώς} \end{cases}$$

Οι παραπάνω κατανομές σχεδιάστηκαν με το ακόλουθο πρόγραμμα Python και φαίνονται ακολούθως :

Η7 ανεξάρτητα ως $p(x_1, x_2) = p(x_1)p(x_2)$ ως ανεξάρτητα κατανομές

και ανεξάρτητα ως $p(x_1, x_2) = p(x_1)p(x_2)$ ως ανεξάρτητα κατανομές
 ανεξάρτητα, οπότε $p(x_1, x_2) = p(x_1)p(x_2)$ ως ανεξάρτητα κατανομές
 ανεξάρτητα, οπότε $p(x_1, x_2) = p(x_1)p(x_2)$ ως ανεξάρτητα κατανομές

analytical_exercises/plots_em.py

```

1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from mpl_toolkits.mplot3d import Axes3D # needed for 3D surface plots
5
6 # -----
7 # 0. Ensure 'plots/' directory exists
8 # -----
9 # If 'plots' does not exist, it is created. If it exists, nothing happens.
10 os.makedirs("plots", exist_ok=True)
11
12 # -----
13 # 1. EM parameter values (SCALE parameterization)
14 # -----
15 # Professor's model:
16 #  $p(x_1 | \theta_1) = (1/\theta_1) * \exp(-x_1 / \theta_1), \quad x_1 \geq 0$ 
17 #  $p(x_2 | \theta_2) = (1/\theta_2), \quad 0 \leq x_2 \leq \theta_2$ 
18 #
19 #  $\theta(0) = (\theta_1(0), \theta_2(0)) = (2, 3)$ 
20 theta0_1 = 2.0 #  $\theta_1(0)$ : scale (mean) before EM
21 theta0_2 = 3.0 #  $\theta_2(0)$ : upper bound of uniform before EM
22
23 #  $\theta(1) = (\theta_1(1), \theta_2(1)) = (7/3, 5)$ 
24 theta1_1 = 7.0 / 3.0 #  $\theta_1(1)$ : scale (mean) after EM
25 theta1_2 = 5.0 #  $\theta_2(1)$ : upper bound of uniform after EM
26
27 # -----
28 # 2. Define x1 and x2 ranges
29 # -----
30 # x1 from 0 to 6 (covers samples 1, 2, 4 and the exponential tail)
31 x1 = np.linspace(0, 6, 200)
32
33 # x2 ranges:
34 # - Before EM:  $[0, \theta_2(0)] = [0, 3]$ 
35 # - After EM :  $[0, \theta_2(1)] = [0, 5]$ 
36 x2_before = np.linspace(0, theta0_2, 150)
37 x2_after = np.linspace(0, theta1_2, 150)
38
39 # Create 2D grids (mesh) of x1 and x2 values for surface plots
40 X1_before, X2_before = np.meshgrid(x1, x2_before)
41 X1_after, X2_after = np.meshgrid(x1, x2_after)
42
43 # -----
44 # 3. Define p(x1, x2) before and after EM
45 # -----
46 # Because  $p(x_1, x_2) = p(x_1) * p(x_2)$ , we compute marginals and multiply.
47
48 # ----- Before EM:  $\theta(0) = (2, 3)$  -----
49 #  $p(x_1 | \theta_1(0)) = (1 / \theta_1(0)) * \exp(-x_1 / \theta_1(0)), \quad x_1 \geq 0$ 
50 p1_before = (1.0 / theta0_1) * np.exp(-X1_before / theta0_1)
51

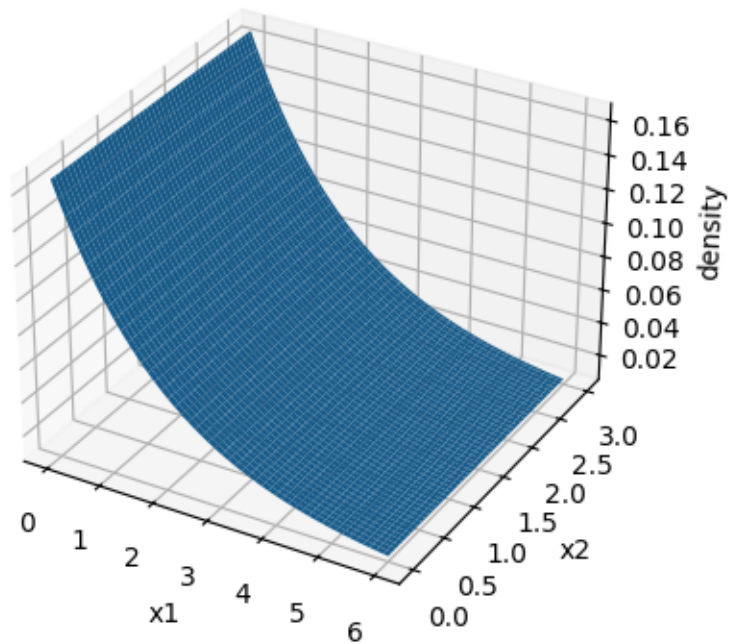
```

```

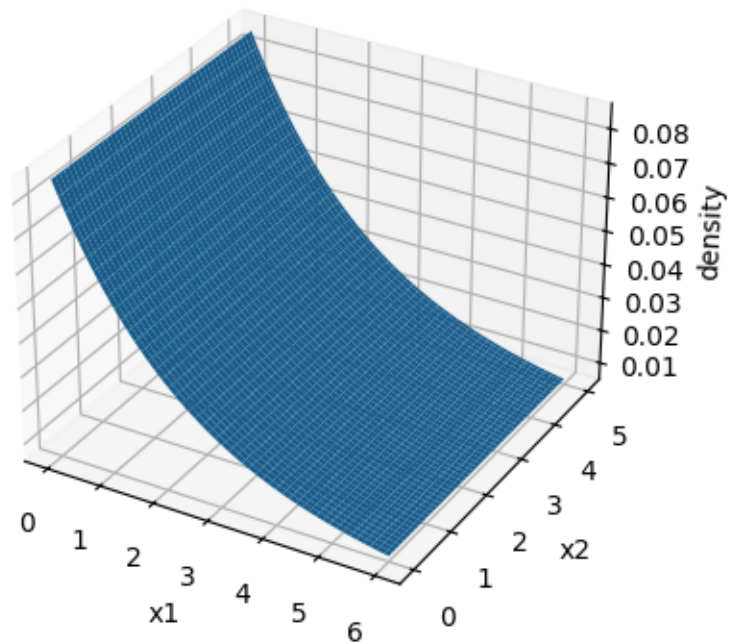
52 #  $p(x_2 \mid \theta_2(0)) = 1 / \theta_2(0)$ ,  $0 \leq x_2 \leq \theta_2(0)$ 
53 # (we already restrict  $x_2\_before$  to  $[0, \theta_2(0)]$ )
54  $p2\_before = (1.0 / \theta_2(0)) * np.ones\_like(X2\_before)$ 
55
56 # Joint density (properly normalized on  $[0, \infty) \times [0, \theta_2(0)]$ )
57  $p\_before = p1\_before * p2\_before$ 
58
59 # ----- After EM:  $\theta(1) = (7/3, 5)$  -----
60 #  $p(x_1 \mid \theta_1(1)) = (1 / \theta_1(1)) * \exp(-x_1 / \theta_1(1))$ ,  $x_1 \geq 0$ 
61  $p1\_after = (1.0 / \theta_1(1)) * np.exp(-X1\_after / \theta_1(1))$ 
62
63 #  $p(x_2 \mid \theta_2(1)) = 1 / \theta_2(1)$ ,  $0 \leq x_2 \leq \theta_2(1)$ 
64  $p2\_after = (1.0 / \theta_2(1)) * np.ones\_like(X2\_after)$ 
65
66 # Joint density (properly normalized on  $[0, \infty) \times [0, \theta_2(1)]$ )
67  $p\_after = p1\_after * p2\_after$ 
68
69 # -----
70 # 4. Plot the two surfaces side by side
71 # -----
72  $fig = plt.figure(figsize=(10, 4))$ 
73
74 # Before EM ( $\theta(0)$ )
75  $ax1 = fig.add\_subplot(1, 2, 1, projection='3d')$ 
76  $ax1.plot\_surface(X1\_before, X2\_before, p\_before)$ 
77  $ax1.set\_title('p(x_1, x_2) at \theta(0) = (2, 3)')$ 
78  $ax1.set\_xlabel('x_1')$ 
79  $ax1.set\_ylabel('x_2')$ 
80  $ax1.set\_zlabel('density')$ 
81
82 # After EM ( $\theta(1)$ )
83  $ax2 = fig.add\_subplot(1, 2, 2, projection='3d')$ 
84  $ax2.plot\_surface(X1\_after, X2\_after, p\_after)$ 
85  $ax2.set\_title('p(x_1, x_2) at \theta(1) = (7/3, 5)')$ 
86  $ax2.set\_xlabel('x_1')$ 
87  $ax2.set\_ylabel('x_2')$ 
88  $ax2.set\_zlabel('density')$ 
89
90  $plt.tight\_layout()$ 
91
92 # -----
93 # 5. Save figure inside 'plots/' directory
94 # -----
95  $output\_path = "plots/EM\_Plots.png"$ 
96  $fig.savefig(output\_path)$ 
97

```

$p(x_1, x_2)$ at $\theta(0) = (2, 3)$



$p(x_1, x_2)$ at $\theta(1) = (7/3, 5)$



5. Bayes Network

TRUE: 1/11/2

TRUE: 1/11/2

(α) Στόχος μας είναι να γράψουμε τη ζητούμενη από κοινού πιθανότητα ως γινόμενο επιμέρους (δεσμευμένων) πιθανοτήτων.

Έστω $P_a(i)$ το σύνολο των γονέων-κόμβων του κόμβου i , δηλαδή των κόμβων με εισερχόμενη ακμή προς τον i . Τότε, με βάση την τριτολογία του δεσμευμένου γραφικού-δικτύου έχουμε:

$$P(a, s, t, l, b, e, x, d) = \prod_{i=1}^n P(x_i | P_a(x_i)) \quad , \text{όπου } x_i \in V \quad (d)$$

Συνεπώς $P(a, s, t, l, b, e, x, d) = P(a) P(s) P(t|a) P(l|s) P(b|s) P(e|t, l) P(x|e) P(d|b, e)$

(β) Θα χρησιμοποιήσουμε d-separation. Έτσι, ένα μονοπάτι μεταξύ δύο κόμβων, δεδομένου ενός συνόλου παρατηρήσιμων κόμβων Z , είναι μηδενικοποιημένο, εάν:

1. Υπάρχει ένας "non-collider" κόμβος στο μονοπάτι, που ανήκει στο Z
2. Υπάρχει ένας "collider" στο μονοπάτι, στο οποίο τόσο ο collider όσο και όλοι του οι απόγονοι δεν ανήκουν στο Z .

Εάν όλα τα μονοπάτια μεταξύ των κόμβων είναι μηδενικοποιημένα, τότε οι κόμβοι αυτοί είναι ανεξάρτητοι, δεδομένου του Z . Συνεπώς, έχουμε:

(b₁) $l \perp\!\!\!\perp b | s$?

Εδώ, $Z = \{s\}$ και έχουμε τα εξής (μη-κατευθυνόμενα) μονοπάτια μεταξύ των b και s :

- i) $l \leftarrow s \rightarrow b$
- ii) $l \rightarrow e \rightarrow d \leftarrow b$

Στην (i) ο s είναι non-collider και $s \in Z \xRightarrow{(1)}$ μηδενικοποιημένο μονοπάτι
 Στην (ii) ο d είναι collider και δεν έχουμε κενό κόμβο στο Z από αυτόν και τους απόγονους του = $\{e\}$, ενώ ο e είναι non-collider και $e \notin Z$. Συνεπώς και αυτό το μονοπάτι είναι μηδενικοποιημένο.

TRUE: 1/11/2

TRUE: 1/11/2

Τελικά: $\boxed{e \parallel b \mid s : \text{TRUE}}$

2. Bayes Network

Μπορούμε να επιβεβαιώσουμε το παραπάνω και αλγεβρικά:

$P(e, b, s) = P(s) P(e \mid s) P(b \mid s)$. (όπου που δεν έχουν και το e και το b)

Έτσι: $P(e, b \mid s) = \frac{P(e, b, s)}{P(s)} = P(e \mid s) P(b \mid s)$, το οποίο είναι ο ορισμός του $e \parallel b \mid s$.

(b₂) $a \parallel s \mid e$?

Εδώ $Z = \{e\}$ και έχουμε τα εξής (μη-κατευθυνόμενα) μονοπάτια μεταξύ a και s :

i) $a \rightarrow t \rightarrow e \leftarrow l \leftarrow s$

ii) $a \rightarrow t \rightarrow e \rightarrow d \leftarrow b \leftarrow s$

Όσοι αφορά το μονοπάτι (i):

t : non-collider και $t \notin Z$
 e : collider και $e \in Z$ και επίσης $x \notin Z$ και $d \notin Z$
 l : non-collider και $l \notin Z$

$\Rightarrow$ μηδοκαρισμένο μονοπάτι από e (συνθήκη 2) και l (συνθήκη 1)

Όσοι αφορά το μονοπάτι (ii):

t : non-collider και $t \notin Z$
 e : non-collider και $e \notin Z$
 d : collider και $a \rightarrow t \rightarrow e \rightarrow d \notin Z$ και $d \notin Z$
 b : non-collider και $b \notin Z$

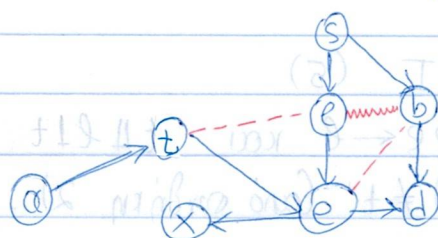
$\Rightarrow$ μηδοκαρισμένο μονοπάτι από d (συνθήκη 2) $d \perp b \mid s$ (p)

Συνεπώς, τόσο το μονοπάτι (i) όσο και το (ii) είναι μηδοκαρισμένα.

Τελικά: $\boxed{a \parallel s \mid e : \text{TRUE}}$

Ομοίως, μπορούμε να το αποδείξουμε και αλγεβρικά.

(c) Αρχικά, ο moralized/triangulated γράφος είναι ο: (προφανώς μη-κατευθυνόμενος)



moralization

triangulization

Ακολούθως, ενισχυαίνουμε πως για να έχει κάποιος κόμβος-τυπή T , πρέπει τουλάχιστον ένας πατέρας του να έχει τυπή T , ενώ για να είναι F πρέπει όλοι του οι γονείς να είναι F . Επίσης, υποθέτουμε τον συμβολισμό:

$$\begin{cases} X : X=T \text{ για λόγους ταχυτύτητας} \\ \bar{X} : X=F \end{cases}$$

Έτσι, οι ζητούμενες πιθανότητες είναι:

(g) $P(x)$

Είρου: $P(x) = P(x|e)P(e) + P(x|\bar{e})P(\bar{e})$ (1)

Όπως: $P(e) = 1 - P(\bar{e}) = 1 - P(\bar{t}, \bar{e})$

Όπως: $t \perp\!\!\!\perp e \Rightarrow P(e) = 1 - P(\bar{t}) \cdot P(\bar{e})$ (2)

Επίσης: $P(t) = P(t|\alpha)P(\alpha) + P(t|\bar{\alpha})P(\bar{\alpha}) = 0.05 \cdot 0.01 + 0.01 \cdot (1 - 0.01) = 0.0104$ (3)

$P(e) = P(e|s)P(s) + P(e|\bar{s})P(\bar{s}) = 0.1 \cdot 0.5 + 0.01 \cdot (1 - 0.5) = 0.055$ (4)

Άρα: $(2) \xrightarrow{(3)} P(e) = 1 - (1 - 0.0104)(1 - 0.055) = 0.0648$

Συνεπώς: $(1) \Rightarrow P(x) = 0.98 \cdot 0.0648 + 0.05 \cdot 0.9352 \approx 0.1103$

Άρα: $\begin{cases} P(x) = 0.1103 \\ P(\bar{x}) = 0.8897 \end{cases}$

(C₂) $P(x|\bar{t})$

Αρχικά, δεδομένου $t=F$ ισχύει: $e=T \Leftrightarrow \ell=T$ (5)

Από condition στο t , τότε: $t \rightarrow e \leftarrow \ell$ και $t \perp \ell$,
αφού e collider και $e \neq t$ και x, d ανεξάρτητοι $e|t$ (από συνθήκη 2).

Συνεπώς:

$$P(e|\bar{t}) = P(e) = 0.055 \text{ και } P(\bar{e}|\bar{t}) = P(\bar{e}) = 0.945$$

$$\text{Επομένως: } P(x|\bar{t}) = \sum_{e_i} P(x|e_i) P(e_i|\bar{t}) = P(x|e) P(e|\bar{t}) + P(x|\bar{e}) P(\bar{e}|\bar{t})$$

$$\text{Όπως: } (5) \Rightarrow P(e|\bar{t}) = P(\ell|\bar{t}) = P(e) = 0.055 \Rightarrow P(\bar{e}|\bar{t}) = 0.945$$

$$\text{Συνεπώς: } P(x|\bar{t}) = 0.98 \cdot 0.55 + 0.05 \cdot 0.945 \approx 0.1012$$

(C₃) $P(t|\bar{x})$

Χρησιμοποιώντας τον κανόνα του Bayes: $P(t|\bar{x}) = \frac{P(\bar{x}|t)P(t)}{P(\bar{x})}$

$$P(\bar{x}) = (100-1) \cdot 100 + 100 \cdot 200 = 109000$$

$$P(t|\bar{x}) = \frac{P(\bar{x}|t)P(t)}{P(\bar{x})} = \frac{(200-1) \cdot 100 + 200 \cdot 10 = 21900}{109000} = \frac{219}{1090}$$

Ηδη έχουμε: $P(\bar{x}) = 0.8897$ και $P(t) = 0.0104$

$$\text{Όπως: } t=T \Rightarrow e=T$$

$$\text{Επομένως: } P(\bar{x}|t) = P(\bar{x}|e) = 0.02$$

$$\text{Άρα: } P(t|\bar{x}) = \frac{0.02 \cdot 0.0104}{0.8897} \approx 2.34 \cdot 10^{-4}$$

(g) $P(t|\bar{x}, d)$

Χρησιμοποιώντας τον κανόνα του Bayes: $P(t|\bar{x}, d) \sim P(\bar{x}, d|t)P(t)$.

Επομένως, χρειαζόμαστε το $P(\bar{x}, d|t_i)$, για $t_i \in \{True, false\}$.

Διακρίνουμε τις περιπτώσεις:

i) $t=T$

Τότε πιθανοσυντελεστής $e=1$. Έχουμε:

$$P(b) = P(b|s)P(s) + P(b|\bar{s})P(\bar{s}) = 0.5 \cdot 0.6 + 0.5 \cdot 0.3 = 0.45$$

Άρα: $P(\bar{b}) = 1 - P(b) = 0.55$

Συνεπώς: $P(d|e) = P(d|e, b)P(b) + P(d|e, \bar{b})P(\bar{b}) = 0.45 \cdot 0.9 + 0.55 \cdot 0.7 \approx 0.79$

Άρα: $P(\bar{x}, d|t) = P(\bar{x}|e)P(d|e) = 0.02 \cdot 0.79 = 0.0158$

ii) $t=F$

Εδώ έχουμε: $P(\bar{x}, d|\bar{t}) = \sum_{s_i, t_i, b_i} P(s_i)P(t_i|s_i)P(b_i|s_i)P(\bar{x}|e=e_i)P(\bar{d}|e=e_i, b_i)$,

για κάθε συνδυασμό των $s_i \in \{0, 1\} \times t_i \in \{0, 1\} \times b_i \in \{0, 1\}$.

Υπογραφή από πράξη προκύπτει:

$$P(\bar{x}, d|\bar{t}) \approx 0.36897$$

Τελικά, από (i) και (ii) έχουμε: $P(t|\bar{x}, d) = \frac{P(\bar{x}, d|t)P(t)}{P(\bar{x}, d|t)P(t) + P(\bar{x}, d|\bar{t})P(\bar{t})} =$

$$= \frac{0.0158 \cdot 0.0104}{0.0158 \cdot 0.0104 + 0.36897 \cdot 0.9896} \approx \frac{0.00016432}{0.36530} \approx 4.50 \cdot 10^{-4}, \text{ που}$$

είναι και το ζητούμενο

6. RNNs

(α) Έχουμε: $S_k = w_{\text{rec}} S_{k-1} + w_x X_k$, $k=1, \dots, T$
 και: $y = S_T$
 Συνάρτηση: $L(y, t) = (t - y)^2 = (t - S_T)^2$ και υποθέτουμε $S_0 = 0$

Επομένως, παίρνουμε: $L = (t - y)^2 = (y - t)^2 = (S_T - t)^2$

$$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial S_T} = 2(S_T - t) = \delta_T \quad (1)$$

Ευρίσκουμε: $\frac{\partial S_k}{\partial S_{k-1}} = w_{\text{rec}}$

Ορίζουμε: $\delta_k = \frac{\partial L}{\partial S_k} = \frac{\partial L}{\partial S_{k+1}} \cdot \frac{\partial S_{k+1}}{\partial S_k} = \delta_{k+1} \cdot w_{\text{rec}}$

Έτσι, έχουμε την αναδρομική σχέση:
$$\begin{cases} \delta_T = 2(S_T - t) \\ \delta_k = \delta_{k+1} \cdot w_{\text{rec}}, \quad k=T-1, \dots, 1 \end{cases}$$

Ισοδύναμα:
$$\begin{cases} \delta_T = 2(S_T - t) \\ \delta_k = \delta_T \cdot w_{\text{rec}}^{T-k} = 2(S_T - t) w_{\text{rec}}^{T-k} \end{cases}$$

Απόφα: $\frac{\partial S_k}{\partial w_x} = X_k$ και $\frac{\partial S_k}{\partial w_{\text{rec}}} = S_{k-1}$

Συνάρτηση:
$$\frac{\partial L}{\partial w_x} = \sum_{k=1}^T \frac{\partial L}{\partial S_k} \cdot \frac{\partial S_k}{\partial w_x} = \sum_{k=1}^T \delta_k X_k$$

$$\frac{\partial L}{\partial w_{\text{rec}}} = \sum_{k=1}^T \frac{\partial L}{\partial S_k} \cdot \frac{\partial S_k}{\partial w_{\text{rec}}} = \sum_{k=1}^T \delta_k S_{k-1}$$

$$810 = (21-1) 100 \cdot 0 = \frac{16}{x \cdot 6} \cdot 100 = \frac{16}{x \cdot 6} \cdot 100$$

$$1 = 0 \cdot 100 - 1 = \frac{16}{x \cdot 6} \cdot 100 = \frac{16}{x \cdot 6} \cdot 100$$

Συνεπώς, οι τελικοί καθόροι αναρρέωσης βαρύνει είναι :

2.4.1.2

$$\begin{cases} w_x^{new} = w_x^{old} - \eta \sum_{k=1}^T \delta_k X_k \\ w_{rec}^{new} = w_{rec}^{old} - \eta \sum_{k=1}^T \delta_k S_{k-1} \end{cases} \quad \text{με} \quad \begin{cases} \delta_T = 2(S_T - t) \\ \delta_k = w_{rec} \delta_{k+1} \end{cases}$$

(b) Εδώ, έχουμε :

$$\begin{cases} T=6 \\ w_{rec}^{(0)} = 1, w_x^{(0)} = 0 \\ t = \# \text{ of 1 in sequence } 101010^6 = 3 \end{cases}$$

Ορίζουμε $\eta = 0.01$ και ναύπρουμε :

Βήμα 1

$$\begin{aligned} S_1 &= 1 \cdot S_0 + 0 \cdot X_1 = 0 \\ S_2 &= 1 \cdot S_1 + 0 = 0 \end{aligned}$$

Ομοίως, $S_3 = S_4 = S_5 = S_6 = 0$

Συνεπώς : $y^{(0)} = S_T = S_6 = 0$ και $L^{(0)} = (t - S_T)^2 = (3 - 0)^2 = 9$

Άρα : $\delta_T = \delta_6 = 2(S_T - t) = 2(0 - 3) = -6$
 $\delta_5 = \delta_6 w_{rec} = -6$

Ομοίως : $\delta_4 = \delta_3 = \delta_2 = \delta_1 = \delta_0 = -6$ (αφού $w_{rec} = 1$)

Συνεπώς :

$$\begin{cases} \frac{\partial L}{\partial w_x} = \sum_{k=1}^6 \delta_k X_k = -6 - 6 - 6 = -18 \\ \frac{\partial L}{\partial w_{rec}} = \sum_{k=1}^6 \delta_k S_{k-1} = 0 \end{cases}$$

Τελικά :

$$\begin{cases} w_x^{(1)} = w_x^{(0)} - \eta \frac{\partial L}{\partial w_x} = 0 - 0.01 \cdot (-18) = 0.18 \\ w_{rec}^{(1)} = w_{rec}^{(0)} - \eta \frac{\partial L}{\partial w_{rec}} = 1 - 0.01 \cdot 0 = 1 \end{cases}$$

• Βήμα 2

Επόμενη

$$S_0 = 0$$

$$S_1 = 1 \cdot 0 + 0.18 \cdot 1 = 0.18$$

$$S_2 = 1 \cdot 0.18 + 0.18 \cdot 0 = 0.18$$

$$S_3 = 0.18 + 0.18 = 0.36$$

$$S_4 = 1 \cdot 0.36 + 0.18 \cdot 0 = 0.36$$

$$S_5 = 1 \cdot 0.36 + 1 \cdot 0.18 = 0.54$$

$$S_6 = 1 \cdot 0.54 + 0 = 0.54$$

Επομένως: $y^{(1)} = S_T - S_6 = 0.54$ και $L^{(1)} = (t - S_T)^2 = (3 - 0.54)^2 \approx 6.0516$
 Παρατηρούμε ότι το σφάλμα μειώθηκε περίπου 30%, μόλις σε 1 βήμα.

Είναι: $\delta_T = \delta_6 = 2(S_6 - t) = 2(0.54 - 3) = -4.92$

Από $w_{rec} = 1 \Rightarrow \delta_k = \delta_6 = -4.92$, $\forall k = 0, \dots, 5$

$$\text{Συνεπώς: } \begin{cases} \frac{\partial L}{\partial w_x} = \sum_{k=1}^6 \delta_k X_k = \delta_1 + \delta_3 + \delta_5 = -14.76 \\ \frac{\partial L}{\partial w_{rec}} = \sum_{k=1}^6 \delta_k S_{k-1} = \delta_6 \sum_{k=1}^6 S_{k-1} = -4.92 \cdot 1.62 \approx -7.9704 \end{cases}$$

$$\text{Τελικά: } \begin{cases} w_{rec}^{(2)} = w_{rec}^{(1)} - \eta \frac{\partial L}{\partial w_{rec}} = 1 - 0.01 \cdot (-7.9704) \approx 1.0797 \\ w_x^{(2)} = w_x^{(1)} - \eta \frac{\partial L}{\partial w_x} = 0.18 - 0.01 \cdot (-14.76) \approx 0.3276 \end{cases}$$

$$w_{rec}^{(3)} = \frac{16}{x_{avg}} \mu - \frac{(2)}{x_{avg}} w = \frac{(3)}{x_{avg}} w$$

$$w_x^{(3)} = \frac{16}{x_{avg}} \mu - \frac{(2)}{x_{avg}} w = \frac{(3)}{x_{avg}} w$$

• Βήμα 3

$$S_0 = 0$$

$$S_1 = 1.0797 \cdot 0 + 0.3276 \cdot 1 = 0.3276$$

$$S_2 = 1.0797 \cdot 0.3276 + 0.3276 \cdot 0 = 0.3537$$

$$S_3 = 0.7095$$

$$S_4 = 0.7661$$

$$S_5 = 1.1547$$

$$S_6 = 1.2467$$

$$0 = 2$$

$$81.0 = 1 \cdot 81.0 + 0 \cdot 1 = 2$$

$$81.0 = 0 \cdot 81.0 + 81.0 \cdot 1 = 2$$

$$25.0 = 81.0 + 81.0 = 2$$

$$25.1.0 = 81.0 + 25.0 \cdot 1 = 2$$

$$42.0 = 81.0 \cdot 1 + 25.0 \cdot 1 = 2$$

$$42.2.0.0 + 42.0 \cdot 1 = 2$$

Συναρτήσεις: $y^{(2)} = S_6 = 1.2467$ και $L^{(2)} = (t - S_6)^2 = (3 - 1.2467)^2 \approx 3.0739$

Το σφάλμα συνεχίζει να μειώνεται δραματικά, αυτή τη φορά κατά ~50%.

Άρα: $\delta_6 = 2(S_6 - t) \approx -3.5065$

Αποσυναρμολόγηση: $\delta_k = w_{rec}^{(2)6-k} \delta_6$ προκύπτουν: $\delta_0 = -5.5552$

$$\delta_1 = -5.1452$$

$$\delta_2 = -4.7653$$

$$\delta_3 = -4.4136$$

$$\delta_4 = -4.0877$$

$$\delta_5 = -3.7860$$

Συναρτήσεις:
$$\left\{ \begin{aligned} \frac{\partial L}{\partial w_k} &= \sum_{x=1}^6 \delta_x X_k \approx 0.12 - 13.3447 = -13.2247 \\ \frac{\partial L}{\partial w_{rec}} &= \sum_{x=1}^6 \delta_x S_{x-1} \approx 8.22 - 12.9718 = -4.7518 \end{aligned} \right.$$

Τελικά:
$$\left\{ \begin{aligned} w_x^{(3)} &= w_x^{(2)} - \eta \frac{\partial L}{\partial w_x} \approx 0.4610 \\ w_{rec}^{(3)} &= w_{rec}^{(2)} - \eta \frac{\partial L}{\partial w_{rec}} \approx 1.2094 \end{aligned} \right.$$

που είναι και το ζητούμενο

(c) Σκοπός του μοντέλου μας είναι να υπολογιστεί τον αριθμό από "1" στην ακολουθία εισόδου, δηλαδή :

$$t = \sum_{k=1}^T x_k$$

Θεωρητικά, εάν επιλέξουμε $w_{rec} = w_x = 1$, τότε : $s_k = s_{k-1} + x_k$ και $s_0 = 0$.
 Δηλαδή, θυμόμαστε την αναδρομική σχέση :

$$s_T = \sum_{k=1}^T x_k$$

Τότε : $L = (t - s_T)^2 = 0$, δηλαδή έχουμε εντοπίσει όλα τα "1" στην είσοδο.
 Αυτό, είναι κάτι καλό ως αποτέλεσμα, ωστόσο επηρεάζει το μοντέλο από το να μάθει, δηλαδή αυτή αντγράφει την είσοδο. Αυτό, γίνεται άμεσα αντιληπτό από την αναδρομική σχέση που εξαγάγαμε προηγουμένως :

$$\delta_k = w_{rec}^{T-k} \delta_T$$

καθώς : $\begin{cases} |w_{rec}| >> 1 \Rightarrow \text{exploding gradients} \\ |w_{rec}| < 1 \Rightarrow \text{vanishing gradients} \end{cases}$

Επομένως, η βέλτιστη αρχικοποίηση είναι $|w_{rec}| \approx 1$ και $|w_x| \approx 1$, ώστε το μοντέλο αφενός να μπορεί να μάθει, αφετέρου να μην έχουμε τα προβλήματα των vanishing/exploding gradients.

Αυτά τα αποτελέσματα - παρατηρήσεις θα αποτελέσουν τη βάση μας για όλα τα επόμενα ερωτήματα.

Εδώ, επιλέγουμε την καλύτερη αρχικοποίηση : $(w_{rec}^{(0)}, w_x^{(0)}) = (1.0, 0.8)$

Εκτελούμε την εκπαίδευση του μοντέλου και παίρνουμε:

$$S_0 = 0$$

$$S_1 = 1 \cdot 0 + 0.8 \cdot 1 = 0.8$$

$$S_2 = 1 \cdot 0.8 + 0.8 \cdot 0 = 0.8$$

$$S_3 = 1 \cdot 0.8 + 0.8 \cdot 1 = 1.6$$

$$S_4 = 1 \cdot 1.6 + 0.8 \cdot 0 = 1.6$$

$$S_5 = 1 \cdot 1.6 + 0.8 \cdot 1 = 2.4$$

$$S_6 = 1 \cdot 2.4 + 0.8 \cdot 0 = 2.4$$

Συνεπώς $L^{(6)} = (t - S_6)^2 = (3 - 2.4)^2 \approx 0.36$ και $S_6 = y^{(6)} = 2.4$

Παρατηρούμε ότι τα σφάλματα αυτά είναι πολύ μικρά και πολύ μικρότερα από το σφάλμα $y^{(2)}$ του 3^{ου} βήματος με τα προηγούμενα βάρη. Συνεπώς, η καλύτερη αυτή αρχικοποίηση βρήκε πολύ.

Επίσης: $\delta_T = \delta_6 = 2(S_T - t) = -1.2$ και $w_{rec}^{(6)} = 1 \Rightarrow \delta_k = -1.2, \forall k=0, \dots, 5$

Είαι:

$$\begin{cases} \frac{\partial L}{\partial w_x} = \sum_{k=1}^6 \delta_k X_k = -1.2(1+1+1) = -3.6 \\ \frac{\partial L}{\partial w_{rec}} = \sum_{k=1}^6 \delta_k S_{k-1} = -1.2 \sum_{k=1}^6 S_{k-1} = -1.2(7.2) = -8.64 \end{cases}$$

Τελικά:

$$\begin{cases} w_x^{(1)} = w_x^{(6)} - \eta \frac{\partial L}{\partial w_x} = 0.8 + 0.01 \cdot 3.6 = 0.836 \\ w_{rec}^{(1)} = w_{rec}^{(6)} - \eta \frac{\partial L}{\partial w_{rec}} = 1 + 0.01 \cdot 8.64 = 1.0864 \end{cases}$$

Τα βάρη παραμένουν πολύ κοντά στο 1 και μάλλον, ύστερα από πράξεις, προκύπτει ότι $L^{(1)} \approx 0.06$.

Συνεπώς, η διαίρεση μιας $w_{opt} = (w_{rec,opt}, w_{x,opt}) = (1, 1)$ ισχύει και παραμιασικά.

(1) Χρησιμοποιήσαμε το γνωστό RNN της άσκησης, ώστε αυτό να εκπαιδεύεται πάνω στην ακολουθία εισόδου 101010, για 30 εποχές, ξεκινώντας από 3 τυχαίες αρχικοποιήσεις βαρών. Το πρόγραμμα αυτό και τα αποτελέσματά μας φαίνονται στις επόμενες σελίδες.

Συμπεραίνοντας με τα αποτελέσματα, παρατηρούμε ακριβώς όσα αναλύσαμε και θεωρητικά. Η συμπεριφορά όλων των w_{rec} και w_x , ανεξαρτήτως αρχικοποίησης είναι κοινή: μεταβαίνουν από μεγάλα σε μικρά, δηλαδή στο ιδανικό μοντέλο, ώστε να μειώσουν το σφάλμα. Όσο, οι αρχικοποιήσεις κοντά στο 1 (seed=2 και seed=20) καταφέρνουν να μειώσουν το σφάλμα αισθητά και σχεδόν να το μηδενίσουν, με την πιο κοντά στο 1 αρχικοποίηση (seed=2) να έχει καλύτερη τόσο αρχική όσο και τελική συμπεριφορά. Αντίθετως, η αρχικοποίηση σε πολύ μικρά βάρη (seed=16) υπέφερε από το πρόβλημα των vanishing gradients, όπως εξηγήσαμε, και έτσι δεν κατάφερε ποτέ να ρίξει ελάχιστα το loss.

(II)

$$\frac{16}{100} W + \frac{16}{100} W = \frac{16}{100}$$

(II)

$$97 = 97 \cdot 10^{-4} + 10^{-4} \cdot 97$$

analytical_exercises/rnn_6d.py

```

1  #!/usr/bin/env python3
2  """
3  Problem 6(d) - Minimal RNN trainer for a fixed sequence 101010.
4
5  RNN model:
6       $s_k = w_{rec} * s_{k-1} + w_x * x_k$ 
7       $y = s_T$ 
8       $L = (t - y)^2$ , where  $t$  = number of 1s in the sequence.
9  """
10
11 import os
12 import random
13 from typing import List, Tuple
14
15
16 # -----
17 # Forward pass: compute all states and final output y
18 # -----
19
20 def forward_rnn(seq: List[int], w_rec: float, w_x: float) -> Tuple[float, List[float]]:
21     """
22     Forward pass for a single sequence.
23
24     Args:
25         seq: list of bits (0/1), e.g. [1, 0, 1, 0, 1, 0]
26         w_rec: recurrent weight
27         w_x: input weight
28
29     Returns:
30         y: final output ( $s_T$ )
31         states: list [ $s_0, s_1, \dots, s_T$ ] for use in backprop
32     """
33     s = 0.0 # initial state  $s_0$ 
34     states = [s] # store  $s_0$ 
35
36     for x in seq:
37         # recurrence:  $s_k = w_{rec} * s_{k-1} + w_x * x_k$ 
38         s = w_rec * s + w_x * x
39         states.append(s)
40
41     y = s # final state is the output
42     return y, states
43
44
45 # -----
46 # Backward pass: BPTT for this simple 1-D RNN
47 # -----
48
49 def backward_rnn(
50     seq: List[int],
51     t: int,

```

```

52     states: List[float],
53     w_rec: float
54 ) -> Tuple[float, float]:
55     """
56     Backpropagation through time for a single sequence.
57
58     Uses:
59          $L = (t - y)^2$ ,  $y = s_T$ 
60          $\delta_T = dL/ds_T = 2(y - t)$ 
61          $\delta_k = w_{rec} * \delta_{k+1}$ 
62          $\partial L / \partial w_x = \sum_k \delta_k x_k$ 
63          $\partial L / \partial w_{rec} = \sum_k \delta_k s_{k-1}$ 
64
65     Args:
66         seq:    input sequence (list of 0/1)
67         t:      target (number of 1s in seq)
68         states: list [s_0, ..., s_T] from forward pass
69         w_rec:  recurrent weight used in forward
70
71     Returns:
72         grad_w_rec, grad_w_x
73     """
74     T = len(seq)
75     y = states[-1]    # s_T
76
77     # delta_T = dL/ds_T
78     delta = 2.0 * (y - t)
79
80     grad_w_rec = 0.0
81     grad_w_x = 0.0
82
83     # Iterate backwards: k = T-1, ..., 0
84     # seq[k] corresponds to time step k+1
85     # states[k] is s_k
86     for k in reversed(range(T)):
87         x_k = seq[k]
88         s_prev = states[k]
89
90         # Grad contributions from time step k+1
91         grad_w_x += delta * x_k
92         grad_w_rec += delta * s_prev
93
94         # Propagate delta backwards in time:  $\delta_k = w_{rec} * \delta_{k+1}$ 
95         delta *= w_rec
96
97     return grad_w_rec, grad_w_x
98
99
100 # -----
101 # Train the RNN for a single random initialization
102 # -----
103
104 def train_one_run(
105     seq: List[int],

```

```

106     t: int,
107     epochs: int,
108     eta: float,
109     init_seed: int
110 ) -> Tuple[float, float, float, float, float, float]:
111     """
112     Train the RNN on a single fixed sequence for a given number of epochs.
113
114     Args:
115         seq:         fixed input sequence
116         t:           target = number of 1s in seq
117         epochs:      number of epochs (here: 30)
118         eta:         learning rate
119         init_seed:   seed for random initialization
120
121     Returns:
122         (w_rec_init, w_x_init, initial_loss,
123          w_rec_final, w_x_final, final_loss)
124     """
125     rng = random.Random(init_seed)
126
127     # Random initialization of weights in a small range
128     w_rec = rng.uniform(-0.5, 0.9)
129     w_x   = rng.uniform(-0.5, 0.9)
130
131     # Compute initial loss before any update
132     y0, states0 = forward_rnn(seq, w_rec, w_x)
133     initial_loss = (t - y0) ** 2
134     w_rec_init, w_x_init = w_rec, w_x
135
136     # Training loop: same sequence, repeated for 'epochs' steps
137     for _ in range(epochs):
138         # Forward pass
139         y, states = forward_rnn(seq, w_rec, w_x)
140         loss = (t - y) ** 2
141
142         # Backward pass (BPTT)
143         grad_wr, grad_wx = backward_rnn(seq, t, states, w_rec)
144
145         # Gradient descent update
146         w_rec -= eta * grad_wr
147         w_x   -= eta * grad_wx
148
149     # Final loss after training
150     y_final, _ = forward_rnn(seq, w_rec, w_x)
151     final_loss = (t - y_final) ** 2
152
153     return w_rec_init, w_x_init, initial_loss, w_rec, w_x, final_loss
154
155
156 # -----
157 # Main: fixed sequence 101010, 30 epochs, save aligned table
158 # -----
159

```

```

160 def main() -> None:
161     # Fixed input sequence 101010
162     seq = [1, 0, 1, 0, 1, 0]
163     t = sum(seq)          # target = number of 1s = 3
164
165     epochs = 15
166     eta = 0.01           # learning rate (can be adjusted)
167
168     # We will use 3 different random initializations
169     init_seeds = [2, 16, 20]
170
171     # Ensure results directory exists
172     os.makedirs("results", exist_ok=True)
173     output_path = os.path.join("results", "rnn_6d_results.txt")
174
175     with open(output_path, "w", encoding="utf-8") as f:
176         # General info header
177         f.write("Problem 6(d) - RNN counting ones\n")
178         f.write(f"Input sequence: {seq}\n")
179         f.write(f"Target (number of 1s): {t}\n")
180         f.write(f"Epochs: {epochs}\n")
181         f.write(f"Learning rate: {eta}\n\n")
182
183         # Table header (aligned columns)
184         header = (
185             f"{'Run':>3} | "
186             f"{'Seed':>4} | "
187             f"{'w_rec_init':>12} | "
188             f"{'w_x_init':>10} | "
189             f"{'initial_loss':>13} | "
190             f"{'w_rec_final':>12} | "
191             f"{'w_x_final':>10} | "
192             f"{'final_loss':>11}\n"
193         )
194         f.write(header)
195         f.write("-" * (len(header) - 1) + "\n") # separator line (minus final \n)
196
197         # One row per run
198         for run_id, seed in enumerate(init_seeds, start=1):
199             (
200                 w_rec_init,
201                 w_x_init,
202                 initial_loss,
203                 w_rec_final,
204                 w_x_final,
205                 final_loss,
206             ) = train_one_run(
207                 seq=seq,
208                 t=t,
209                 epochs=epochs,
210                 eta=eta,
211                 init_seed=seed,
212             )
213

```

```
214         row = (  
215             f"{run_id:>3} | "  
216             f"{seed:>4} | "  
217             f"{w_rec_init:12.6f} | "  
218             f"{w_x_init:10.6f} | "  
219             f"{initial_loss:13.6f} | "  
220             f"{w_rec_final:12.6f} | "  
221             f"{w_x_final:10.6f} | "  
222             f"{final_loss:11.6f}\\n"  
223         )  
224         f.write(row)  
225  
226     print(f"Results written to: {output_path}")  
227  
228  
229 if __name__ == "__main__":  
230     main()  
231
```

Problem 6(d) - RNN counting ones
Input sequence: [1, 0, 1, 0, 1, 0]
Target (number of 1s): 3
Epochs: 15
Learning rate: 0.01

Run	Seed	w_rec_init	w_x_init	initial_loss	w_rec_final	w_x_final	final_loss
1	2	0.838448	0.826958	2.180193	1.031838	0.903391	0.000351
2	16	0.006132	0.172673	8.993648	0.185997	0.248387	8.715173
3	20	0.767896	0.460756	5.357076	1.123954	0.681514	0.001931

(e-bonus-1)

Στο δικό μας μοντέλο, το loss είναι: $L = (t - s_T)^2$, όπου t ο αριθμός των 1 στην ακολουθία εισόδου. Έτσι, το L υπολογίζεται μόνο μία φορά στο τέλος (βήμα T). Αλλάει, στο βιβλίο θεωρούν ένα RNN με έξοδο o_t σε κάθε βήμα t και σήμα y_t . Συνεπώς, το συνολικό σφάλμα εκεί είναι:

$$L(x_{1:T}, y_{1:T}, w_h, w_o) = \frac{1}{T} \sum_{t=1}^T \ell(y_t, o_t) \quad (I)$$

δηλαδή υπάρχει επιμέρους σφάλμα σε κάθε βήμα.

Συνεπώς, εμείς πρέπει να κάνουμε την τροποποίηση:

$$L = \frac{1}{T} \sum_{k=1}^T (u_k - s_k)^2 \quad (II)$$

Έτσι, οι (I) και (II) είναι αλυσίδες.

Όσο αφορά τα βάρη και τα gradients, στο βιβλίο κάνουν unroll το RNN στον χρόνο και κάνουν backpropagation στον unrolled graph σε κάθε βήμα t , δηλαδή:

$$\frac{\partial L}{\partial h_t} = W_h^T \frac{\partial L}{\partial h_{t+1}} + W_{oh}^T \frac{\partial L}{\partial o_t} \quad (III)$$

Συνεπώς, έχουμε τον αναδρομικό όρο και τον local-loss όρο.

Έτσι, προσθέτουμε και στη δική μας αναδρομική σχέση τον όρο αυτό και παίρνουμε:

$$\delta_k = w_{rec} \delta_{k+1} + 2(s_k - u_k) \quad (IV)$$

Έτσι, με βάση τις (II) και (IV) θα εκπαιδεύσουμε το νέο μας μοντέλο.

(β-bonus-2)

Υποποιήσαμε το παραπάνω μοντέλο (όπως περιγράφηκε) και ο κώδικας φαίνεται - μαζί με τα αποτελέσματα - στην επόμενη σελίδα. Κρατήσαμε όλες τις παραμέτρους σταθερές, για να συγκρίνουμε.

Όπως παρατηρούμε από τα αποτελέσματα, τα κύρια σημεία είναι τα ίδια. Δηλαδή, οι "καλές" αρχικοποιήσεις βελτιώνονται πολύ, ενώ η "κακή" λιγότερο. Αυτό που έχει ιδιαίτερο ενδιαφέρον είναι:

i) Τα τελικά βάρη όλων των αρχικοποιήσεων είναι πιο κοντά στο 1 από ό,τι με το προηγούμενο μοντέλο $\Rightarrow$ καλύτερο αποτέλεσμα

ii) Η κακή αρχικοποίηση βελτιώθηκε σημαντικά περισσότερο

iii) Το τελικό σφάλμα δεν συγκρίνεται άμεσα (αφού έχει άλλο ζήτο πλέον), αλλά η ποιότητα της εκπαίδευσης φαίνεται από την εγγύτητα των τελικών βαρών στο 1, δηλαδή το ιδανικό μοντέλο. Έτσι, τα βάρη είναι πάντα καλύτερα στο νέο μοντέλο.

Συνοψίζοντας, η instance-based εκπαίδευση και ενημέρωση $\forall t$, φαίνεται να είναι πιο αποτελεσματική από την καθολική για το συγκεκριμένο πρόβλημα.

analytical_exercises/rnn_6f.py

```

1  #!/usr/bin/env python3
2  """
3  Problem 6(f) - Bonus-2
4  RNN trainer for a fixed sequence 101010, using BPTT with a loss at every time step.
5
6  RNN model (same recurrence as before):
7      s_k = w_rec * s_{k-1} + w_x * x_k
8
9  But now:
10     • At each time step k, the "desired" output is the cumulative number of ones so far:
11         u_k = sum_{i=1}^k x_i
12     • We interpret the hidden state as the output at each step, y_k = s_k.
13     • The total loss is the average over time:
14         L = (1/T) * sum_{k=1}^T (u_k - s_k)^2
15
16  We implement BPTT with this loss (see 6(e) and the D2L link).
17  """
18
19  import os
20  import random
21  from typing import List, Tuple
22
23
24  # -----
25  # Forward pass: compute all states s_0, ..., s_T
26  # -----
27
28  def forward_rnn(seq: List[int], w_rec: float, w_x: float) -> List[float]:
29      """
30      Forward pass for a single sequence.
31
32      Args:
33          seq: list of bits (0/1), e.g. [1, 0, 1, 0, 1, 0]
34          w_rec: recurrent weight
35          w_x: input weight
36
37      Returns:
38          states: list [s_0, s_1, ..., s_T] for use in backprop
39      """
40      s = 0.0          # initial state s_0
41      states = [s]     # store s_0
42
43      for x in seq:
44          # recurrence: s_k = w_rec * s_{k-1} + w_x * x_k
45          s = w_rec * s + w_x * x
46          states.append(s)
47
48      return states
49
50
51  # -----

```

```

52 # Loss + Backward pass (BPTT with per-time-step loss)
53 # -----
54
55 def loss_and_backward_rnn(
56     seq: List[int],
57     states: List[float],
58     w_rec: float
59 ) -> Tuple[float, float, float]:
60     """
61     Compute the average loss over all time steps and the gradients using BPTT.
62
63     Model and loss:
64          $s_k = w_{\text{rec}} * s_{k-1} + w_x * x_k$ 
65          $u_k$  = cumulative count of ones up to time  $k$ 
66          $L = (1/T) * \sum_{k=1}^T (u_k - s_k)^2$ 
67
68     Let:
69          $\delta_k = dL/ds_k$ 
70
71     Then (scalar version of the D2L formulas):
72          $\text{local}_k = dL_k/ds_k = (2/T) * (s_k - u_k)$ 
73          $\delta_k = \text{local}_k + w_{\text{rec}} * \delta_{k+1}$ 
74     and
75          $\partial L / \partial w_x = \sum_k \delta_k * x_k$ 
76          $\partial L / \partial w_{\text{rec}} = \sum_k \delta_k * s_{k-1}$ 
77
78     Args:
79         seq: input sequence (list of 0/1), length T
80         states: list  $[s_0, \dots, s_T]$  from forward pass
81         w_rec: recurrent weight used in forward
82
83     Returns:
84         loss: average MSE over all time steps
85         grad_w_rec:  $dL/dw_{\text{rec}}$ 
86         grad_w_x:  $dL/dw_x$ 
87     """
88     T = len(seq)
89     # Extract  $s_1, \dots, s_T$ ;  $\text{states}[0] = s_0$ 
90     s_list = states[1:]
91
92     # Compute cumulative targets  $u_k$  and loss
93     cumulative = 0
94     targets = [] #  $u_k$  for  $k=1..T$ 
95     loss = 0.0
96
97     for k in range(T):
98         cumulative += seq[k] #  $u_k$  = number of ones up to step  $k+1$ 
99         u_k = cumulative
100         targets.append(u_k)
101
102         err = s_list[k] - u_k #  $s_k - u_k$ 
103         loss += err * err
104
105     loss /= T # average over time

```

```

106
107     # Backward pass (BPTT)
108     grad_w_rec = 0.0
109     grad_w_x = 0.0
110     delta_next = 0.0                #  $\delta_{T+1} = 0$  (no future contribution)
111
112     # Go backwards: k = T-1,...,0    (step index k corresponds to time k+1)
113     for k in reversed(range(T)):
114         s_k = s_list[k]              #  $s_{k+1}$  in time notation
115         s_prev = states[k]           #  $s_k$  in time notation
116         x_k = seq[k]                 #  $x_{k+1}$ 
117         u_k = targets[k]             # target up to step k+1
118
119         # Local gradient of  $L_k$  wrt  $s_k$ :  $(2/T)*(s_k - u_k)$ 
120         local_grad = (2.0 / T) * (s_k - u_k)
121
122         # Total  $\delta_k$  = local term + recurrent term from future
123         delta = local_grad + w_rec * delta_next
124
125         # Accumulate parameter gradients
126         grad_w_x += delta * x_k
127         grad_w_rec += delta * s_prev
128
129         # Pass gradient to previous step
130         delta_next = delta
131
132     return loss, grad_w_rec, grad_w_x
133
134
135 # -----
136 # Train the RNN for a single random initialization (new model)
137 # -----
138
139 def train_one_run(
140     seq: List[int],
141     epochs: int,
142     eta: float,
143     init_seed: int
144 ) -> Tuple[float, float, float, float, float, float]:
145     """
146     Train the RNN on a single fixed sequence for a given number of epochs,
147     using the new per-time-step loss and BPTT.
148
149     Args:
150         seq:          fixed input sequence
151         epochs:       number of epochs (here: 15)
152         eta:          learning rate
153         init_seed:    seed for random initialization
154
155     Returns:
156         (w_rec_init, w_x_init, initial_loss,
157          w_rec_final, w_x_final, final_loss)
158     """
159     rng = random.Random(init_seed)

```

```

160
161     # Random initialization of weights in a given range
162     w_rec = rng.uniform(-0.5, 0.9)
163     w_x   = rng.uniform(-0.5, 0.9)
164
165     # Compute initial loss before any update (with new loss definition)
166     states0 = forward_rnn(seq, w_rec, w_x)
167     initial_loss, _, _ = loss_and_backward_rnn(seq, states0, w_rec)
168     w_rec_init, w_x_init = w_rec, w_x
169
170     # Training loop: same sequence, repeated for 'epochs' steps
171     for _ in range(epochs):
172         # Forward pass
173         states = forward_rnn(seq, w_rec, w_x)
174
175         # Loss and gradients via BPTT (per-time-step loss)
176         loss, grad_wr, grad_wx = loss_and_backward_rnn(seq, states, w_rec)
177
178         # Gradient descent update
179         w_rec -= eta * grad_wr
180         w_x   -= eta * grad_wx
181
182     # Final loss after training
183     states_final = forward_rnn(seq, w_rec, w_x)
184     final_loss, _, _ = loss_and_backward_rnn(seq, states_final, w_rec)
185
186     return w_rec_init, w_x_init, initial_loss, w_rec, w_x, final_loss
187
188
189 # -----
190 # Main: fixed sequence 101010, 15 epochs, save aligned table
191 # -----
192
193 def main() -> None:
194     # Fixed input sequence 101010
195     seq = [1, 0, 1, 0, 1, 0]
196
197     epochs = 15
198     eta = 0.01          # learning rate
199
200     # 3 different random initializations (as in your version)
201     init_seeds = [2, 16, 20]
202
203     # Ensure results directory exists
204     os.makedirs("results", exist_ok=True)
205     output_path = os.path.join("results", "rnn_6f_results.txt")
206
207     with open(output_path, "w", encoding="utf-8") as f:
208         # General info header
209         f.write("Problem 6(f) - RNN counting ones with BPTT (per-time-step loss)\n")
210         f.write(f"Input sequence: {seq}\n")
211         f.write(f"Epochs: {epochs}\n")
212         f.write(f"Learning rate: {eta}\n\n")
213

```

```
214 # Table header (aligned columns)
215 header = (
216     f"{'Run':>3} | "
217     f"{'Seed':>4} | "
218     f"{'w_rec_init':>12} | "
219     f"{'w_x_init':>10} | "
220     f"{'initial_loss':>13} | "
221     f"{'w_rec_final':>12} | "
222     f"{'w_x_final':>10} | "
223     f"{'final_loss':>11}\n"
224 )
225 f.write(header)
226 f.write("-" * (len(header) - 1) + "\n") # separator line
227
228 # One row per run
229 for run_id, seed in enumerate(init_seeds, start=1):
230     (
231         w_rec_init,
232         w_x_init,
233         initial_loss,
234         w_rec_final,
235         w_x_final,
236         final_loss,
237     ) = train_one_run(
238         seq=seq,
239         epochs=epochs,
240         eta=eta,
241         init_seed=seed,
242     )
243
244     row = (
245         f"{run_id:>3} | "
246         f"{seed:>4} | "
247         f"{w_rec_init:12.6f} | "
248         f"{w_x_init:10.6f} | "
249         f"{initial_loss:13.6f} | "
250         f"{w_rec_final:12.6f} | "
251         f"{w_x_final:10.6f} | "
252         f"{final_loss:11.6f}\n"
253     )
254     f.write(row)
255
256     print(f"Results written to: {output_path}")
257
258
259 if __name__ == "__main__":
260     main()
261
```

Problem 6(f) - RNN counting ones with BPTT (per-time-step loss)
Input sequence: [1, 0, 1, 0, 1, 0]
Epochs: 15
Learning rate: 0.01

Run	Seed	w_rec_init	w_x_init	initial_loss	w_rec_final	w_x_final	final_loss
1	2	0.838448	0.826958	0.787480	1.028084	0.936431	0.002524
2	16	0.006132	0.172673	4.334102	0.102092	0.441046	3.785678
3	20	0.767896	0.460756	2.363265	1.109755	0.771253	0.035703